

هوش مصنوعی و کارگاه

دانشکده ریاضی و علوم کامپیوتر دانشگاه صنعتی امیرکبیر



مدرس کلاس: مهدی قطعی

استاد گروه علوم کامپیوتر

www.aut.ac.ir

مدرس کارگاه: بهنام یوسفی مهر

دانشجوی دکتری علوم کامپیوتر

Artificial Intelligence & Workshop

- Unsupervised and Supervised Learning
Or
- Supervised & Unsupervised Learning

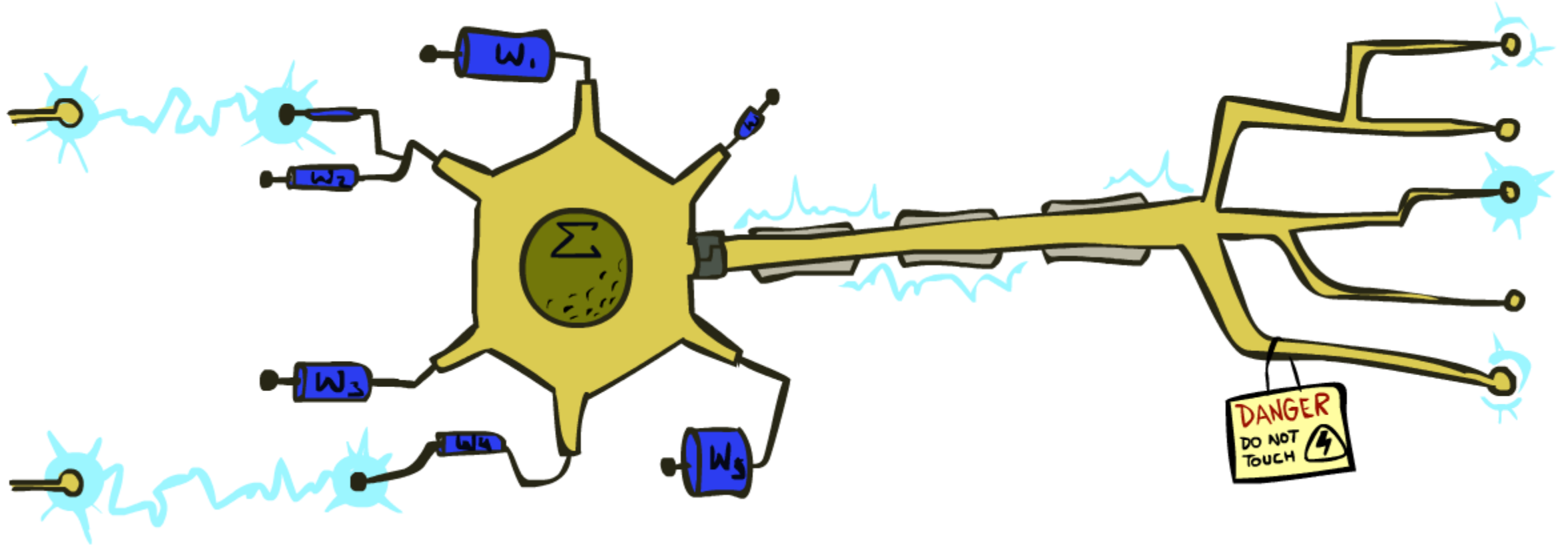
Unsupervised and Supervised Learning

- At one and a half months, the baby perceives three-dimensional round objects, for example, a ball.
- At three months, the child is able to distinguish the face not only of his mother, but also of those who are in constant contact with him - father, grandmother, nanny.
- In the first few months, your baby learns how to interpret the emotion in your voice and how to communicate through smiles.
- Around 6 months, babies can recognize their name, and by 12 months they'll start to understand simple, spoken commands.



Starting Idea

Supervised & Unsupervised Learning



Anca Dragan

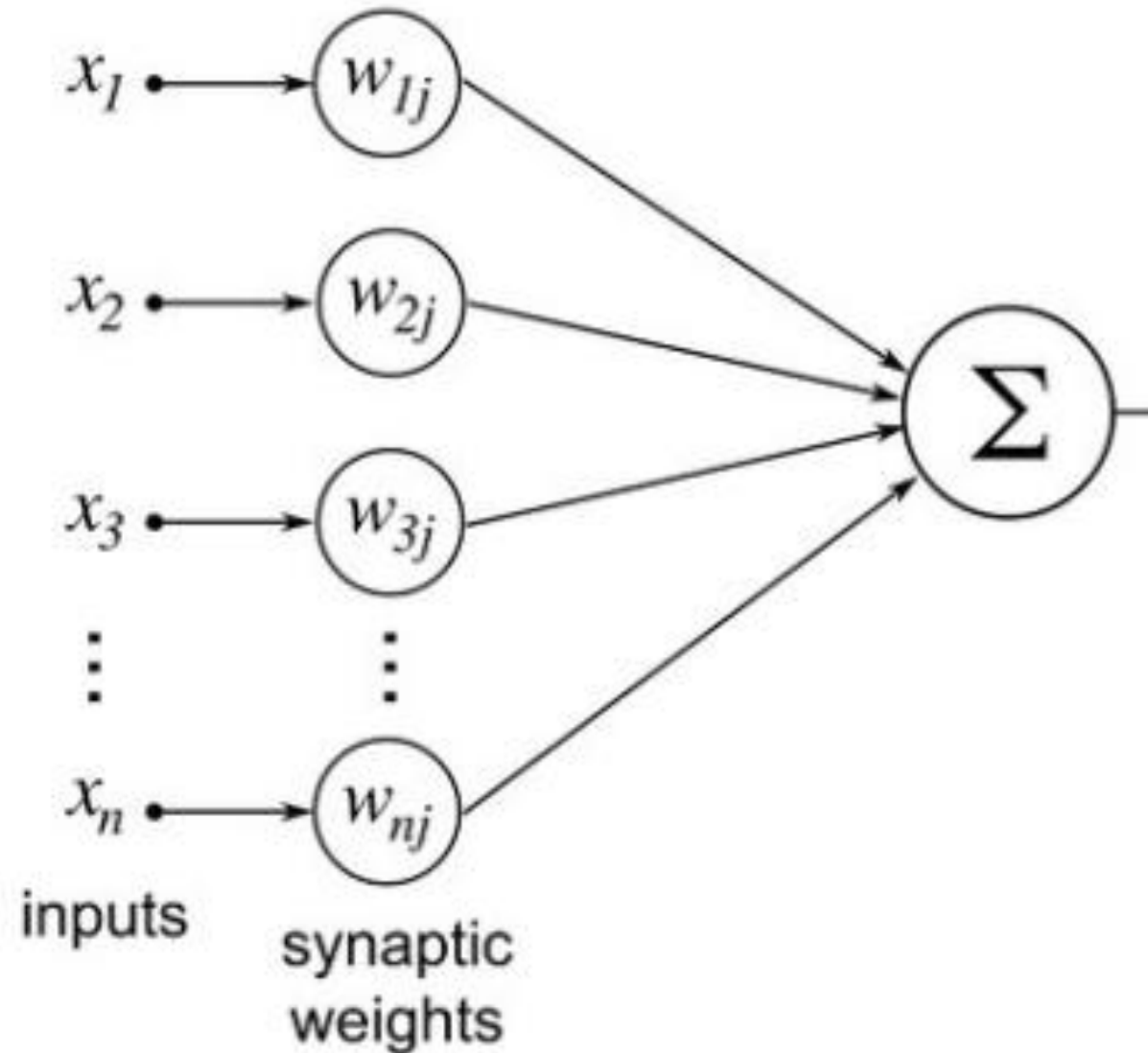
University of California, Berkeley

What we discussed: Probabilistic Inference

- Probabilistic inference: compute a desired probability from other known probabilities (e.g. conditional from joint)
- We generally compute conditional probabilities
 - $P(\text{on time} \mid \text{no reported accidents}) = 0.90$
 - These represent the agent's *beliefs* given the evidence
- Probabilities change with new evidence:
 - $P(\text{on time} \mid \text{no accidents, 5 a.m.}) = 0.95$
 - $P(\text{on time} \mid \text{no accidents, 5 a.m., raining}) = 0.80$
 - Observing new evidence causes *beliefs to be updated*



Probabilistic Inference with new evidence



Inference by Enumeration

- General case:

- Evidence variables: $E_1 \dots E_k = e_1 \dots e_k$
 - Query* variable: Q
 - Hidden variables: $H_1 \dots H_r$
- $$\left. \begin{array}{l} E_1 \dots E_k = e_1 \dots e_k \\ Q \\ H_1 \dots H_r \end{array} \right\} \begin{array}{l} X_1, X_2, \dots X_n \\ \text{All variables} \end{array}$$

- We want:

* Works fine with multiple query variables, too

$$P(Q|e_1 \dots e_k)$$

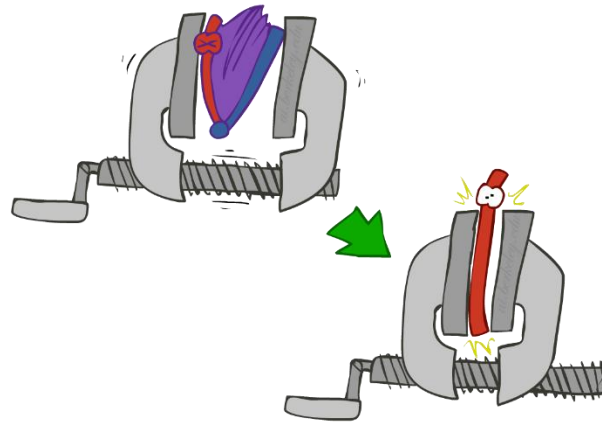
- Step 1: Select the entries consistent with the evidence



x	P(x)
-3	0.05
-1	0.25
0	0.07
1	0.2
5	0.01

2 0.15

- Step 2: Sum out H to get joint of Query and evidence



- Step 3: Normalize

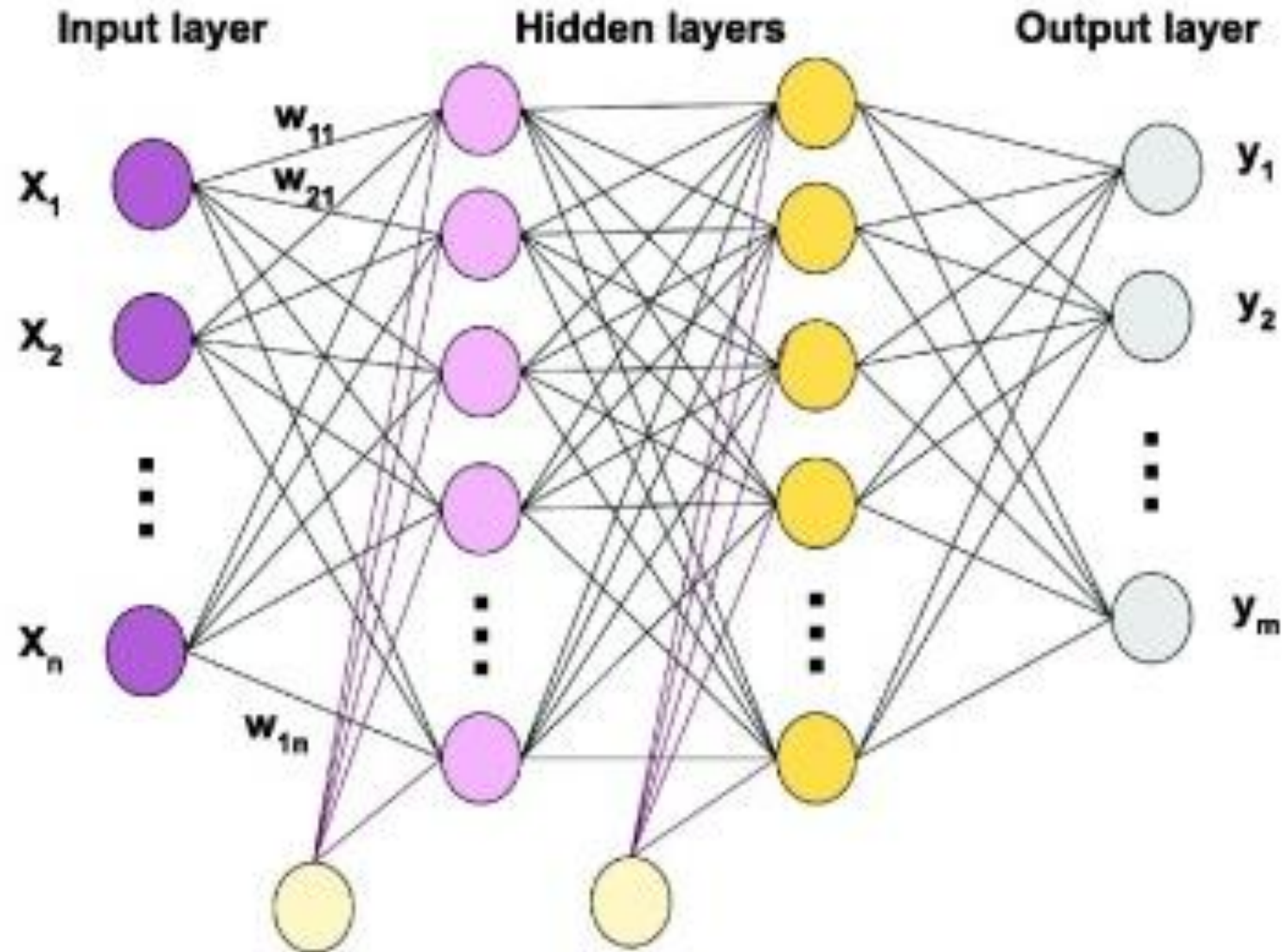
$$\times \frac{1}{Z}$$

$$Z = \sum_q P(Q, e_1 \dots e_k)$$

$$P(Q|e_1 \dots e_k) = \frac{1}{Z} P(Q, e_1 \dots e_k)$$

$$P(Q, e_1 \dots e_k) = \sum_{h_1 \dots h_r} \underbrace{P(Q, h_1 \dots h_r, e_1 \dots e_k)}_{X_1, X_2, \dots X_n}$$

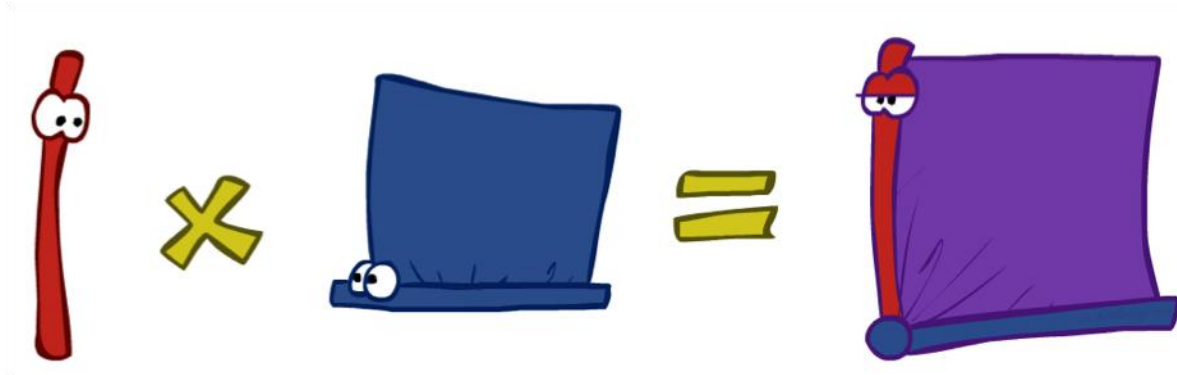
Inference by Enumeration



The Product Rule

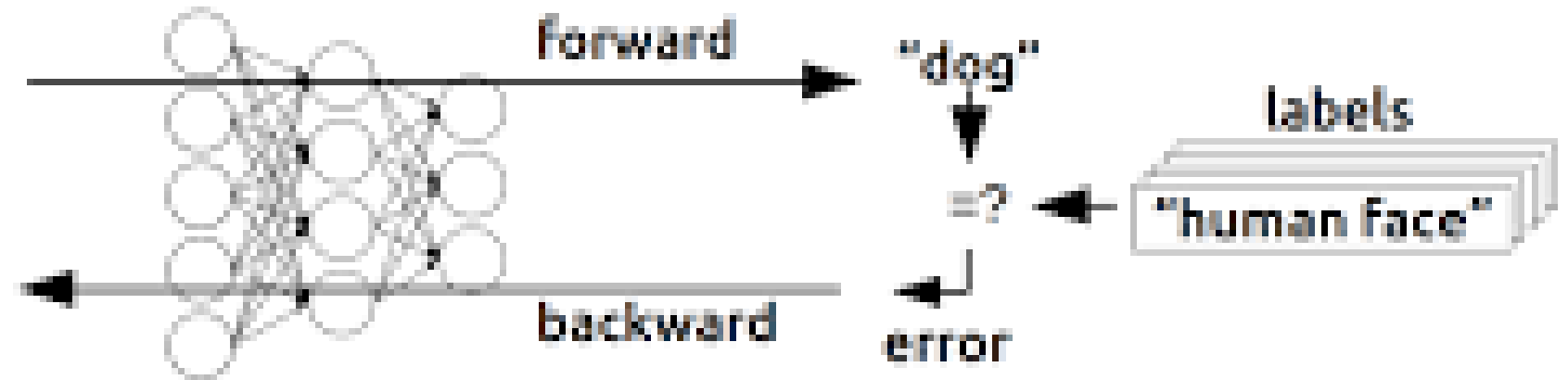
- Sometimes have conditional distributions but want the joint

$$P(y)P(x|y) = P(x, y) \quad \longleftrightarrow \quad P(x|y) = \frac{P(x, y)}{P(y)}$$

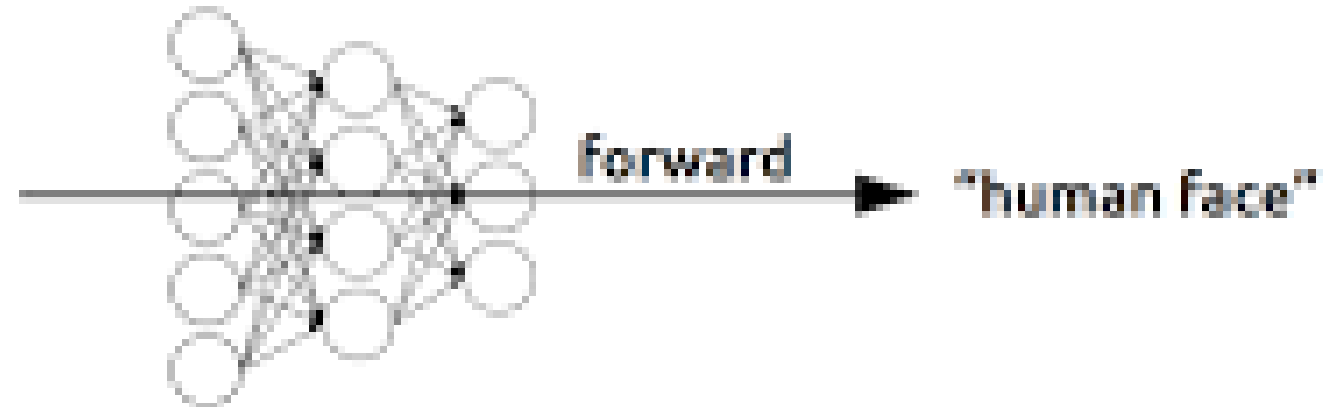


The Product Rule

Training



Inference



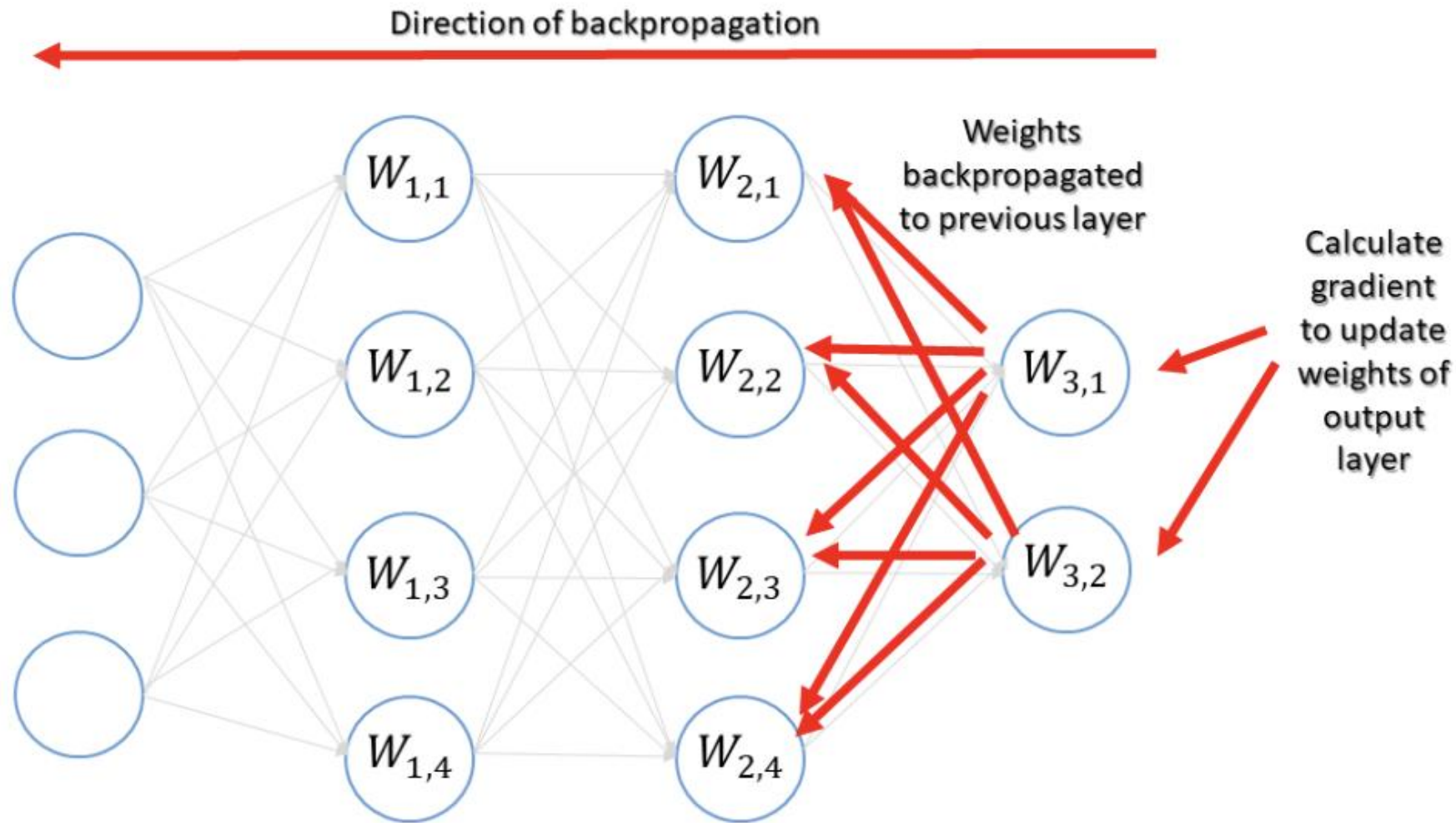
The Chain Rule

- More generally, can always write any joint distribution as an incremental product of conditional distributions

$$P(x_1, x_2, x_3) = P(x_1)P(x_2|x_1)P(x_3|x_1, x_2)$$

$$P(x_1, x_2, \dots, x_n) = \prod_i P(x_i|x_1 \dots x_{i-1})$$

The Chain Rule



Bayes' Rule

- Two ways to factor a joint distribution over two variables:

$$P(x, y) = P(x|y)P(y) = P(y|x)P(x)$$

That's my rule!

- Dividing, *we get*:

$$P(x|y) = \frac{P(y|x)P(x)}{P(y)}$$

- Why is this at all helpful?
 - Lets us build one conditional from its reverse
 - Often one conditional is tricky but the other one is simple
- In the running for most important AI equation!



Inference with Bayes' Rule

- Example: Diagnostic probability from causal probability:

posterior
probability

$$P(\text{cause}|\text{effect}) = \frac{P(\text{effect}|\text{cause})P(\text{cause})}{P(\text{effect})}$$

Prior
probability

Effect
probability

- Example:

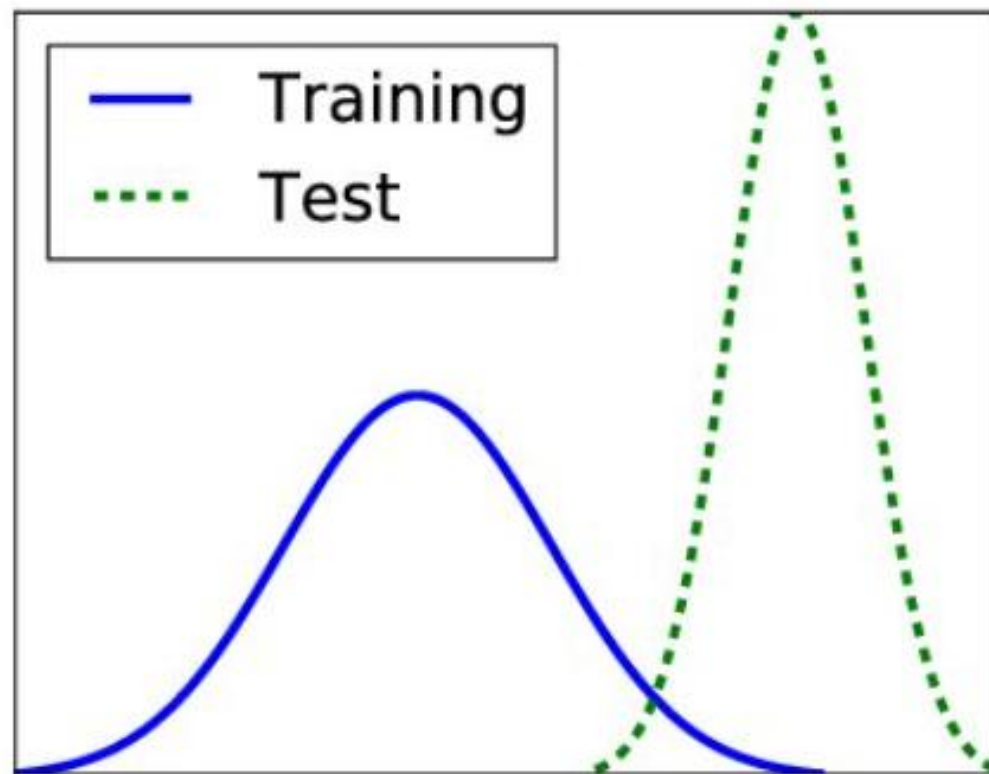
- M: meningitis, S: stiff neck

$$\left. \begin{aligned} P(+m) &= 0.0001 \\ P(+s|+m) &= 0.8 \\ P(+s|-m) &= 0.01 \end{aligned} \right\} \text{Example gives}$$

$$P(+m|+s) = \frac{P(+s|+m)P(+m)}{P(+s)} = \frac{P(+s|+m)P(+m)}{P(+s|+m)P(+m) + P(+s|-m)P(-m)} = \frac{0.8 \times 0.0001}{0.8 \times 0.0001 + 0.01 \times 0.999}$$

- Note: posterior probability of meningitis still very small
- Note: you should still get stiff necks checked out! Why?

Inference with Bayes' Rule

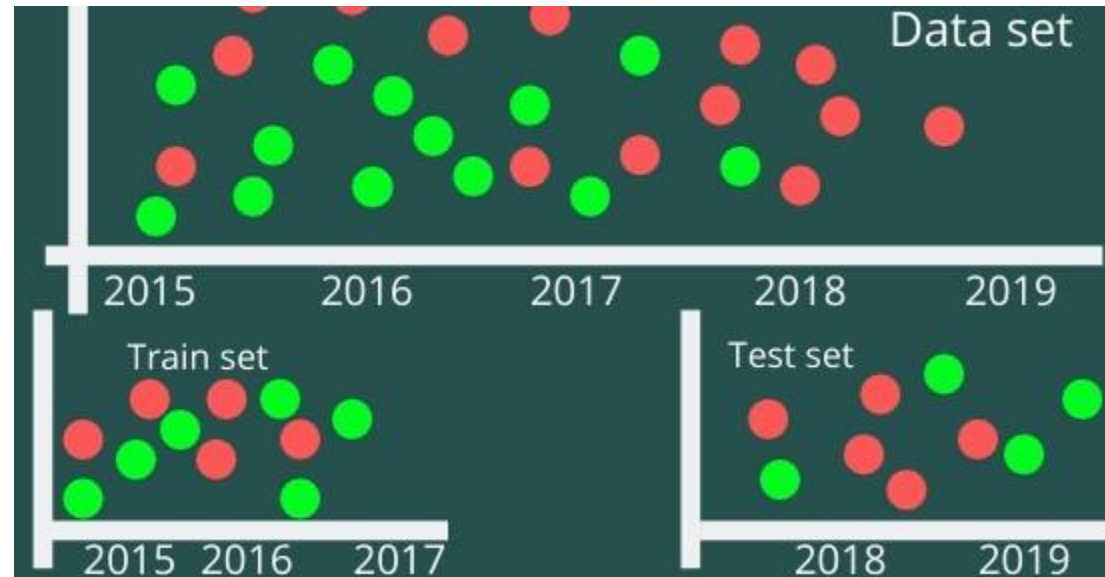


Example of covariate shift: training and test data having different distributions.

Ref: On the Reduction of Biases in Big Data Sets for the Detection of Irregular Power Usage

January 2018, DOI: [10.48550/arXiv.1801.05627](https://doi.org/10.48550/arXiv.1801.05627)

Inference with Bayes' Rule



Time based splitting and determining if Train & Test data come from the same distribution

Ref: <https://www.linkedin.com/pulse/time-based-splitting-determining-train-test-data-come-manraj-chalokia/>

Ref: <https://encord.com/blog/train-val-test-split/>

- Properly splitting your machine learning datasets into training, validation, and test sets is essential for building robust and accurate models.



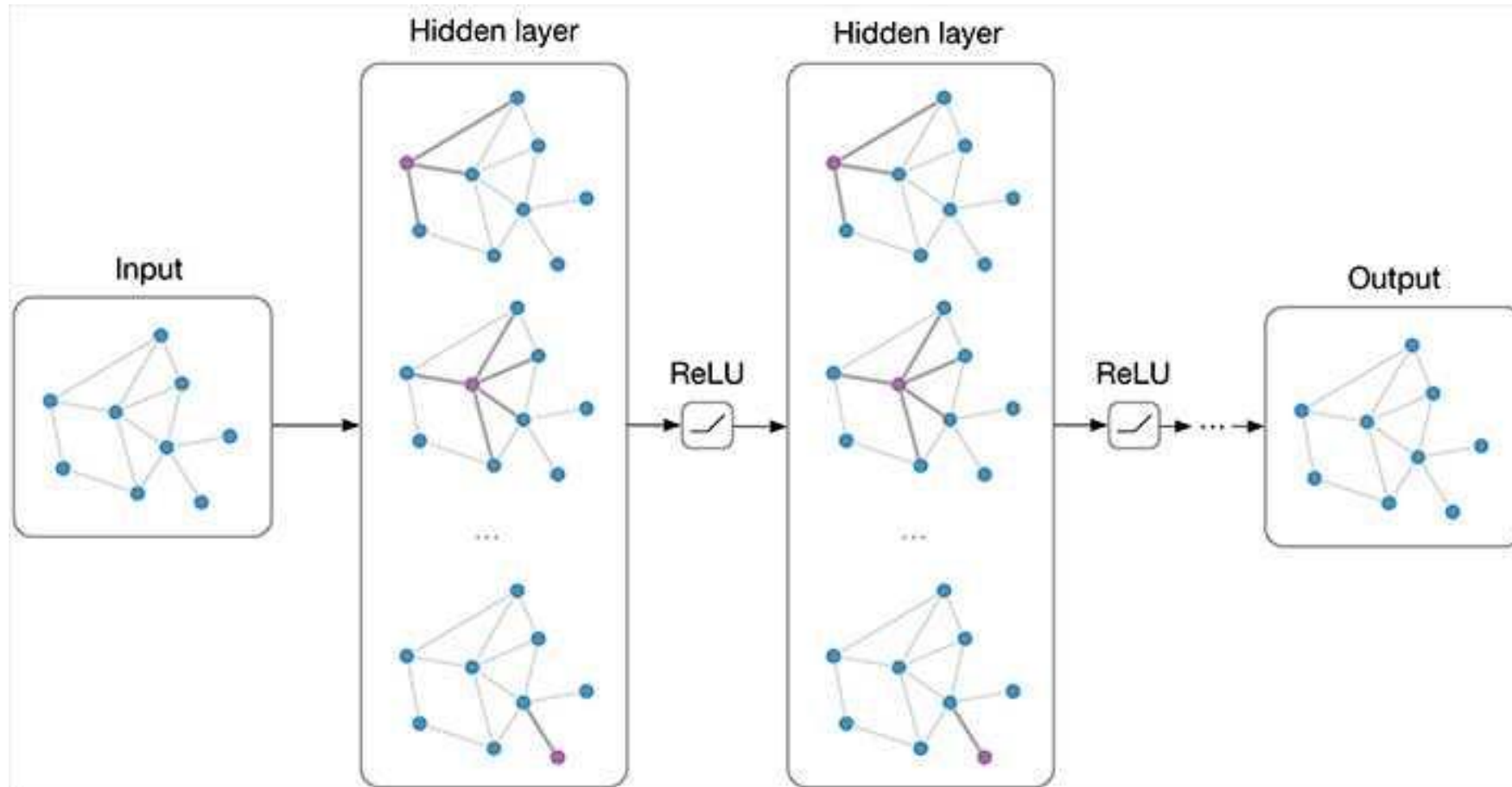
- It explores the differences between these sets, highlights three effective splitting methods, identifies common mistakes to avoid, and explains how to use Encord for managing dataset splits in computer vision projects.
- The train-validation-test split is fundamental to the development of robust and reliable machine learning models.

Probabilistic Graphical Model (PGM)

- It is a rich framework for **encoding probability distributions over complex domains**: joint (multivariate) distributions over large numbers of random variables that interact with each other.
- These representations sit at the intersection of statistics and computer science, relying on concepts from **probability theory, graph algorithms, machine learning**, and more.
- They are the basis for the state-of-the-art methods in a wide variety of applications, such as medical diagnosis, image understanding, speech recognition, natural language processing, and many, many more. They are also a foundational tool in formulating many machine learning problems.

Graph Neural Networks

Ref: <https://tkipf.github.io/graph-convolutional-networks/>



See and enjoy:

Lapenna, Michela, and Caterina De Bacco. "How do Probabilistic Graphical Models and Graph Neural Networks Look at Network Data?." *arXiv preprint arXiv:2506.11869* (2025).

Bayesian Networks

- A Bayesian network (also known as a Bayes network, Bayes net, belief network, or decision network) is a probabilistic graphical model that represents a set of variables and their conditional dependencies via a directed acyclic graph (DAG).
- Bayesian networks are ideal for taking an event that occurred and predicting the likelihood that any one of several possible known causes was the contributing factor.
- For example, a Bayesian network could represent the probabilistic relationships between diseases and symptoms. Given symptoms, the network can be used to compute the probabilities of the presence of various diseases.

Bayesian Networks

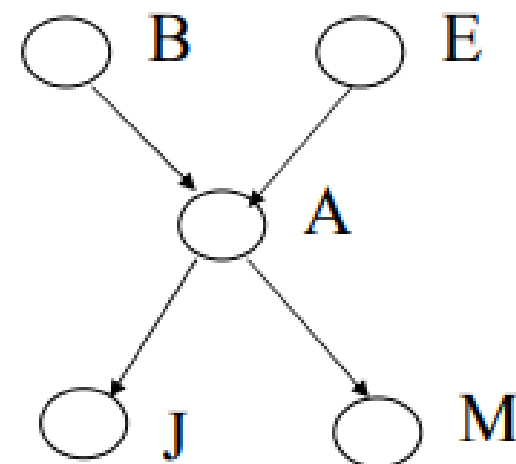
Two components: $B = (S, \Theta_S)$

- **Directed acyclic graph**
 - Nodes correspond to random variables
 - (Missing) links encode independences
- **Parameters**
 - Local conditional probability distributions for every variable-parent configuration

$$\mathbf{P}(X_i \mid pa(X_i))$$

Where:

$pa(X_i)$ - stand for parents of X_i

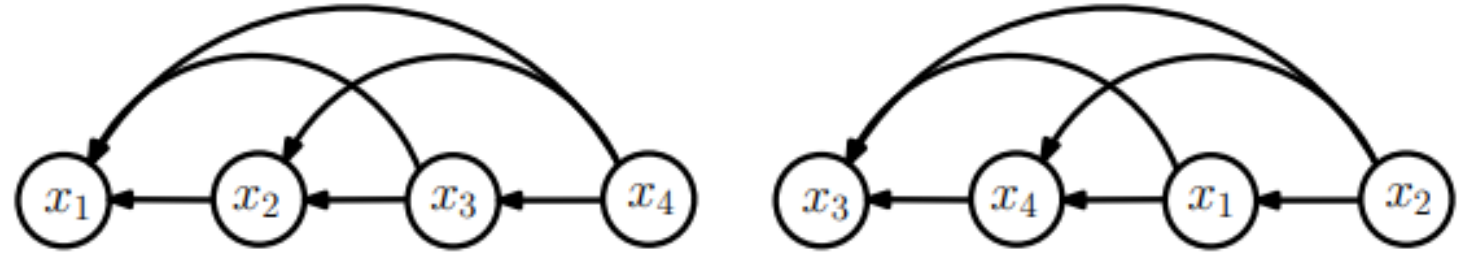


$\mathbf{P}(A|B,E)$

B	E	T	F
T	T	0.95	0.05
T	F	0.94	0.06
F	T	0.29	0.71
F	F	0.001	0.999

Bayesian Networks

- Different Factorizations



- Two factorizations of four variables:

$$p(x_1, x_2, x_3, x_4) = p(x_1 \mid x_2, x_3, x_4)p(x_2 \mid x_3, x_4)p(x_3 \mid x_4)p(x_4)$$

$$p(x_1, x_2, x_3, x_4) = p(x_3 \mid x_1, x_2, x_4)p(x_4 \mid x_1, x_2)p(x_1 \mid x_2)p(x_2)$$

- Any distribution can be written in such a cascade form as a belief network (using chain rule)
- With independence assumptions the factorization often becomes simpler

Markov Model

- A Markov model is a stochastic model used to model pseudo-randomly changing systems.
- It is assumed that **future states depend only on the current state**, not on the events that occurred before it (that is, it assumes the Markov property).
- Generally, this assumption enables reasoning and computation with the model that would otherwise be intractable.
- For this reason, in the fields of predictive modelling and probabilistic forecasting, it is desirable for a given model to exhibit the Markov property.

Markov Model

- Markov chains are used for several business applications, including:
 - predicting customer **brand switching** for marketing
 - predicting **how long people will remain** in their jobs
 - predicting **time to failure** of a machine in manufacturing
 - forecasting the **future price of a stock** in finance
- Markov analysis is also used in natural language processing (NLP) and in machine learning.
 - For NLP, a Markov chain can be used to **generate a sequence of words** that form a complete sentence.
 - For machine learning, Markov decision processes are used to represent **reward in reinforcement learning**.

Markov Model

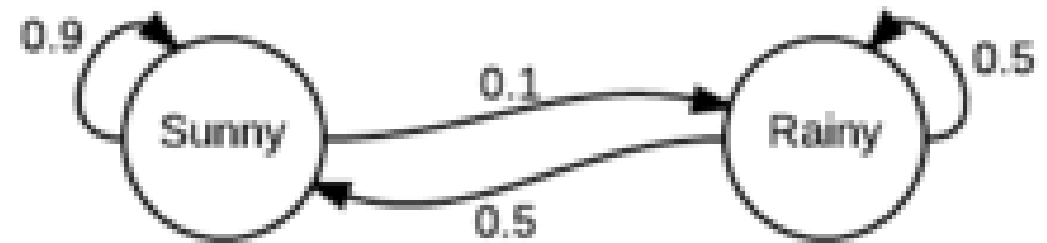
The weather on day 0 (today) is known to be sunny. This is represented by an initial state vector in which the "sunny" entry is 100%, and the "rainy" entry is 0%:

$$\mathbf{x}^{(0)} = [1 \quad 0]$$

The weather on day 1 (tomorrow) can be **predicted by multiplying the state vector from day 0 by the transition matrix**:

$$\mathbf{x}^{(1)} = \mathbf{x}^{(0)} P = [1 \quad 0] \begin{bmatrix} 0.9 & 0.1 \\ 0.5 & 0.5 \end{bmatrix} = [0.9 \quad 0.1]$$

Thus, there is a 90% chance that day 1 will also be sunny. The weather on day 2 (the day after tomorrow) can be predicted in the same way, from the state vector we computed for day 1:



$$\mathbf{x}^{(2)} = \mathbf{x}^{(1)} P = \mathbf{x}^{(0)} P^2 = [1 \quad 0] \begin{bmatrix} 0.9 & 0.1 \\ 0.5 & 0.5 \end{bmatrix}^2 = [0.86 \quad 0.14]$$

or

$$\mathbf{x}^{(2)} = \mathbf{x}^{(1)} P = [0.9 \quad 0.1] \begin{bmatrix} 0.9 & 0.1 \\ 0.5 & 0.5 \end{bmatrix} = [0.86 \quad 0.14]$$

Steady state of the weather $\mathbf{q} = \lim_{n \rightarrow \infty} \mathbf{x}^{(n)}$

Artificial Intelligence & Workshop

- Unsupervised and Supervised Learning
Or
- Supervised & Unsupervised Learning

Different kinds of learning...

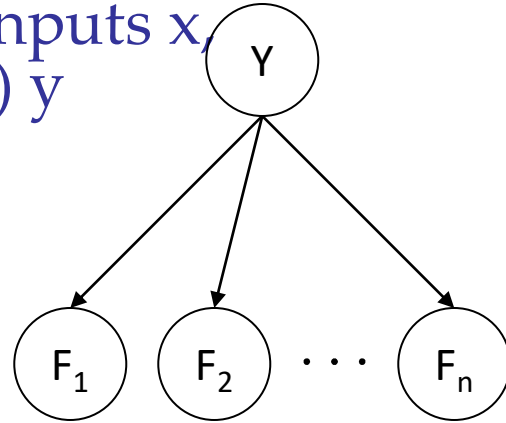
- **Supervised learning:**
 - Someone gives us examples and the right answer for those examples
 - We have to predict the right answer for unseen examples
- **Unsupervised learning:**
 - We see examples but get no feedback
 - We need to find patterns in the data
- **Reinforcement learning:**
 - We take actions and get rewards
 - Have to learn how to get high rewards

Supervised & Unsupervised Learning

- Supervised Learning
 - Error Function Minimization
 - Maximum likelihood
- Unsupervised Learning
 - Similarity Measure
- Reinforcement learning
 - Reward Score

Supervised Learning

- Classification: given inputs x , predict labels (classes) y

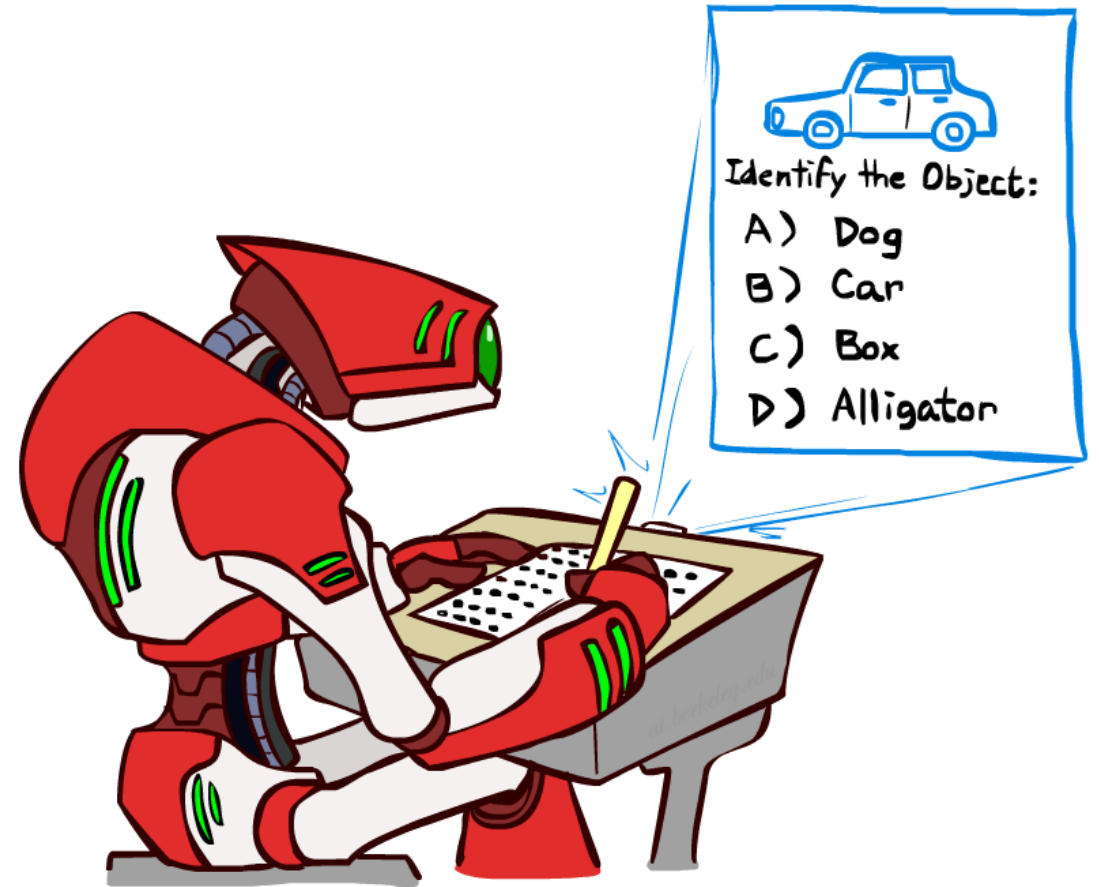


- Naïve Bayes

$$P(Y|F_{0,0} \dots F_{15,15}) \propto P(Y) \prod_{i,j} P(F_{i,j}|Y)$$

- Supervised Learning

- Parameter estimation:
 - Maximum Likelihood Estimation (MLE), Maximum A Posteriori (MAP) Estimation
- Training set, held-out (validation) set, test set

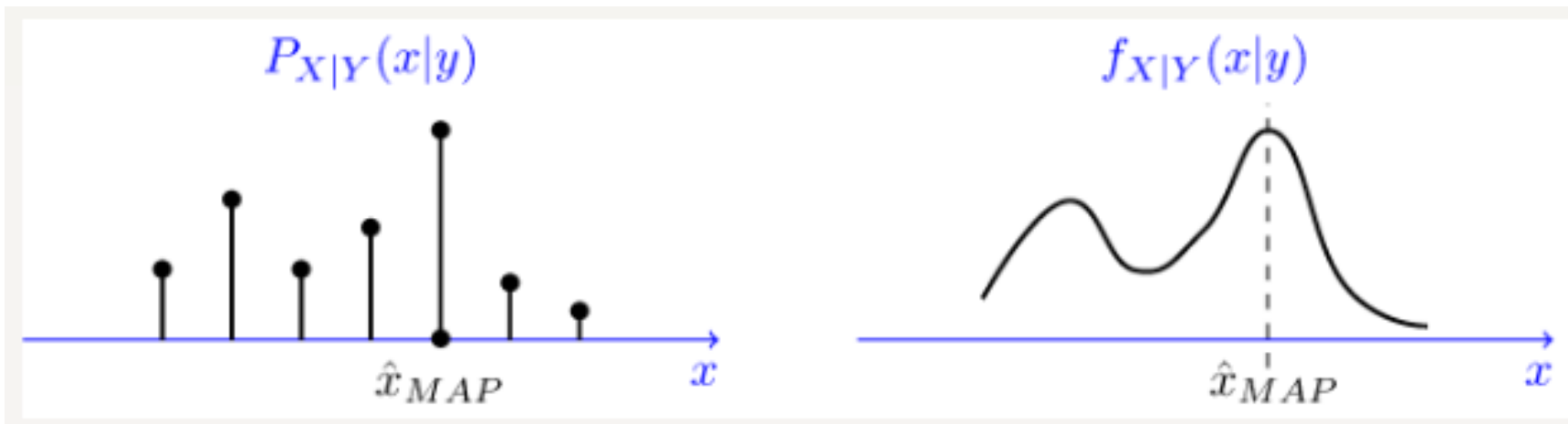


Maximum Likelihood Estimation (MLE)

- It is a method of estimating the parameters of an assumed probability distribution, given some observed data.
- This is achieved by maximizing a likelihood function so that, under the assumed statistical model, the observed data is most probable.
- The point in the parameter space that maximizes the likelihood function is called the maximum likelihood estimate.
- The logic of maximum likelihood is both intuitive and flexible, and as such the method has become a dominant means of statistical inference.
- For independent random variables:
$$f_n(\mathbf{y}; \theta) = \prod_{k=1}^n f_k^{\text{univar}}(y_k; \theta)$$

Maximum A Posteriori (MAP)

- The posterior distribution, $f_{X|Y}(x|y)$ (or $P_{X|Y}(x|y)$), contains all the knowledge about the unknown quantity X . Therefore, we can use the posterior distribution to find point or interval estimates of X .
- One way to obtain a point estimate is to choose the value of X that maximizes the posterior PDF. This is called the ***maximum a posteriori (MAP) estimation***.



Supervised Learning

- Explainable Methods

- Decision Tree
- Random Forest
- Case-Base Reasoning
- Frequent Patterns Based Classification

- Black-Box Methods

- Multi-Layer Perceptron
- Extreme Learning Machines
- Radial Bases Functions (EBF) Networks
- Deep Networks

Example of supervised learning: classification

- We lend money to people
- We have to predict whether they will pay us back or not
- People have various (say, binary) features:
 - do we know their Address? do they have a Criminal record? high Income? Educated? Old? Unemployed?
- We see examples: (Y = paid back, N = not)
 - +a, -c, +i, +e, +o, +u: Y
 - a, +c, -i, +e, -o, -u: N
 - +a, -c, +i, -e, -o, -u: Y
 - a, -c, +i, +e, -o, -u: Y
 - a, +c, +i, -e, -o, -u: N
 - a, -c, +i, -e, -o, +u: Y
 - +a, -c, -i, -e, +o, -u: N
 - +a, +c, +i, -e, +o, -u: N
- Next person is +a, -c, +i, -e, +o, -u. Will we get paid back?

Classification...

- We want some hypothesis h that predicts whether we will be paid back
 - +a, -c, +i, +e, +o, +u: Y
 - a, +c, -i, +e, -o, -u: N
 - +a, -c, +i, -e, -o, -u: Y
 - a, -c, +i, +e, -o, -u: Y
 - a, +c, +i, -e, -o, -u: N
 - a, -c, +i, -e, -o, +u: Y
 - +a, -c, -i, -e, +o, -u: N
 - +a, +c, +i, -e, +o, -u: N
- Lots of possible hypotheses: will be paid back if...
 - Income is high (*wrong on 2 occasions in training data*)
 - Income is high and no Criminal record (*always right in training data*)
 - (Address is known AND ((NOT Old) OR Unemployed)) OR ((NOT Address is known) AND (NOT Criminal Record)) (*always right in training data*)
- Which one seems best? Anything better?

Classification...

- We want some hypothesis h that predicts whether we will be paid back

+a, -c, +i, +e, +o, +u: Y

-a, +c, -i, +e, -o, -u: N

+a, -c, +i, -e, -o, -u: Y

-a, -c, +i, +e, -o, -u: Y

-a, +c, +i, -e, -o, -u: N

-a, -c, +i, -e, -o, +u: Y

+a, -c, -i, -e, +o, -u: N

+a, +c, +i, -e, +o, -u: N

- Lots of possible hypotheses: will be paid back if...
 - Income is high (*wrong on 2 occasions in training data*)
 - Income is high and no Criminal record (*always right in training data*)
 - (Address is known AND ((NOT Old) OR Unemployed)) OR ((NOT Address is known) AND (NOT Criminal Record)) (*always right in training data*)
- Which one seems best? Anything better?

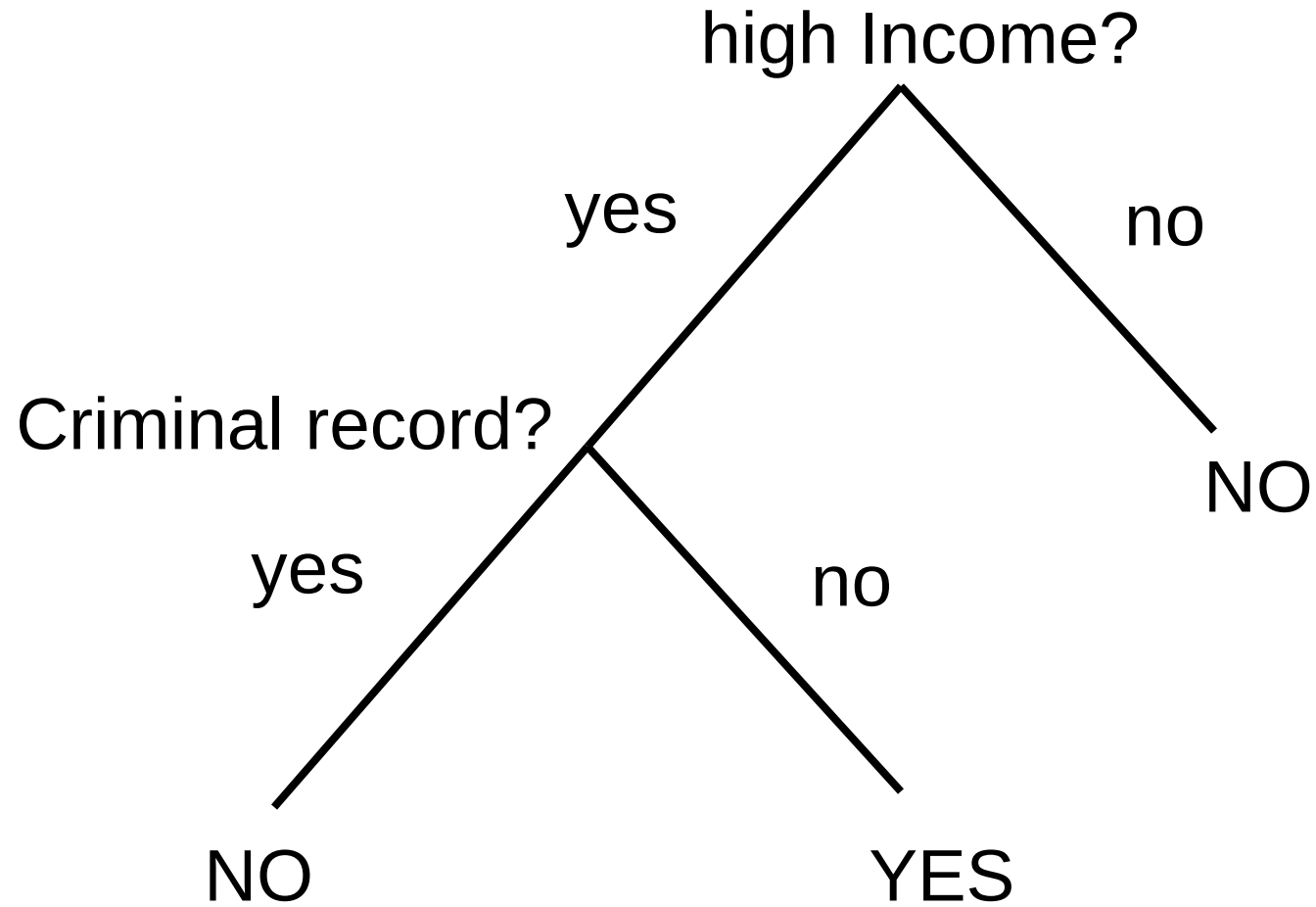
Classification...

- We want some hypothesis h that predicts whether we will be paid back
 - +a, -c, +i, +e, +o, +u: Y
 - a, +c, -i, +e, -o, -u: N
 - +a, -c, +i, -e, -o, -u: Y
 - a, -c, +i, +e, -o, -u: Y
 - a, +c, +i, -e, -o, -u: N
 - a, -c, +i, -e, -o, +u: Y
 - +a, -c, -i, -e, +o, -u: N
 - +a, +c, +i, -e, +o, -u: N
- Lots of possible hypotheses: will be paid back if...
 - Income is high (*wrong on 2 occasions in training data*)
 - Income is high and no Criminal record (*always right in training data*)
 - (Address is known AND ((NOT Old) OR Unemployed)) OR ((NOT Address is known) AND (NOT Criminal Record)) (*always right in training data*)
- Which one seems best? Anything better?

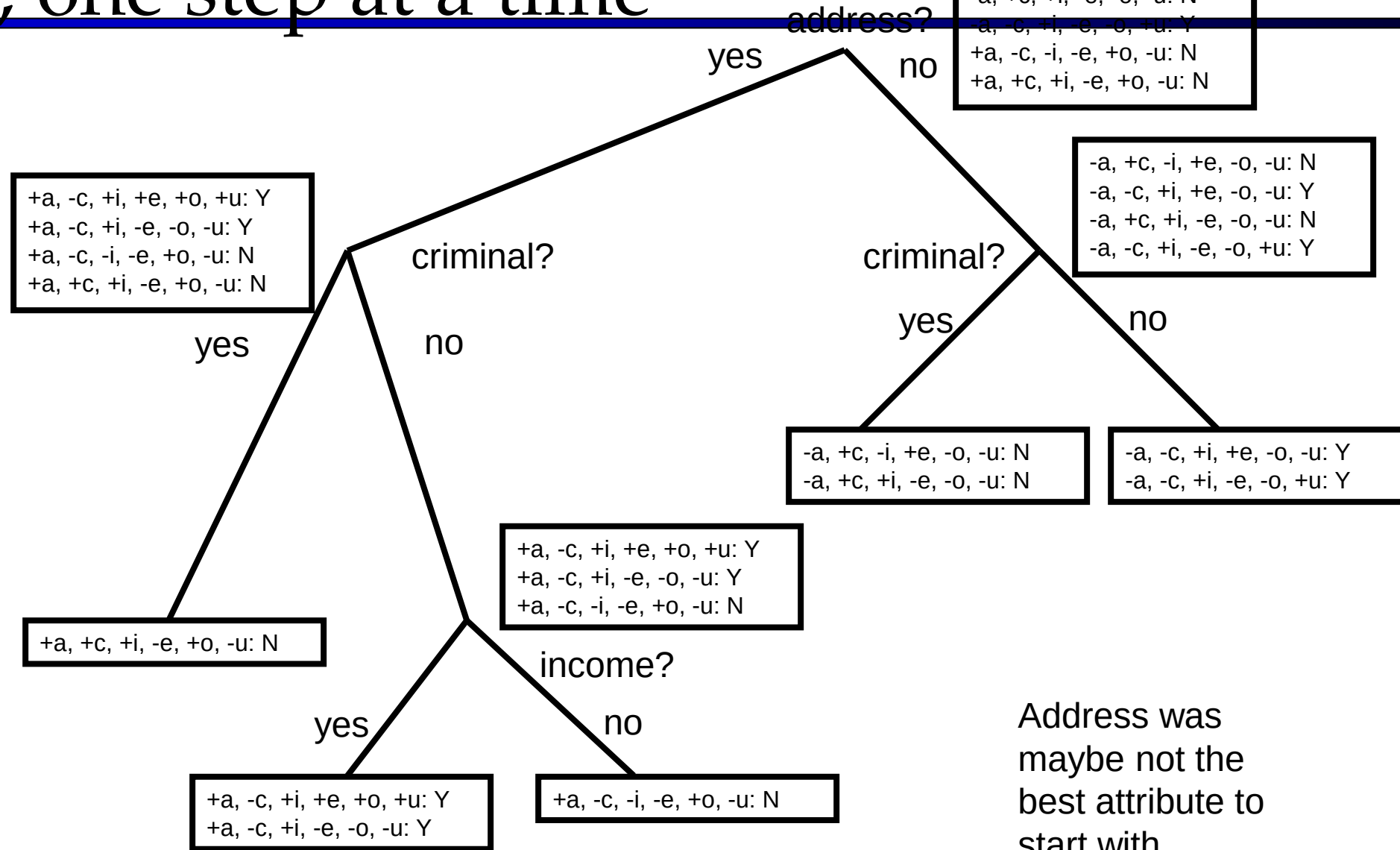
Occam's Razor: The problem-solving principle that recommends searching for explanations constructed with the smallest possible set of elements.

- Occam's razor: *simpler hypotheses tend to generalize to future data better*
- Intuition: given limited training data,
 - it is likely that there is some **complicated** hypothesis that **is not actually good** but that happens to **perform well on the training data**
 - it is less likely that there is a **simple** hypothesis that is not actually good but that happens to perform well on the training data
 - There are fewer simple hypotheses
- **Computational learning theory** studies this in much more depth

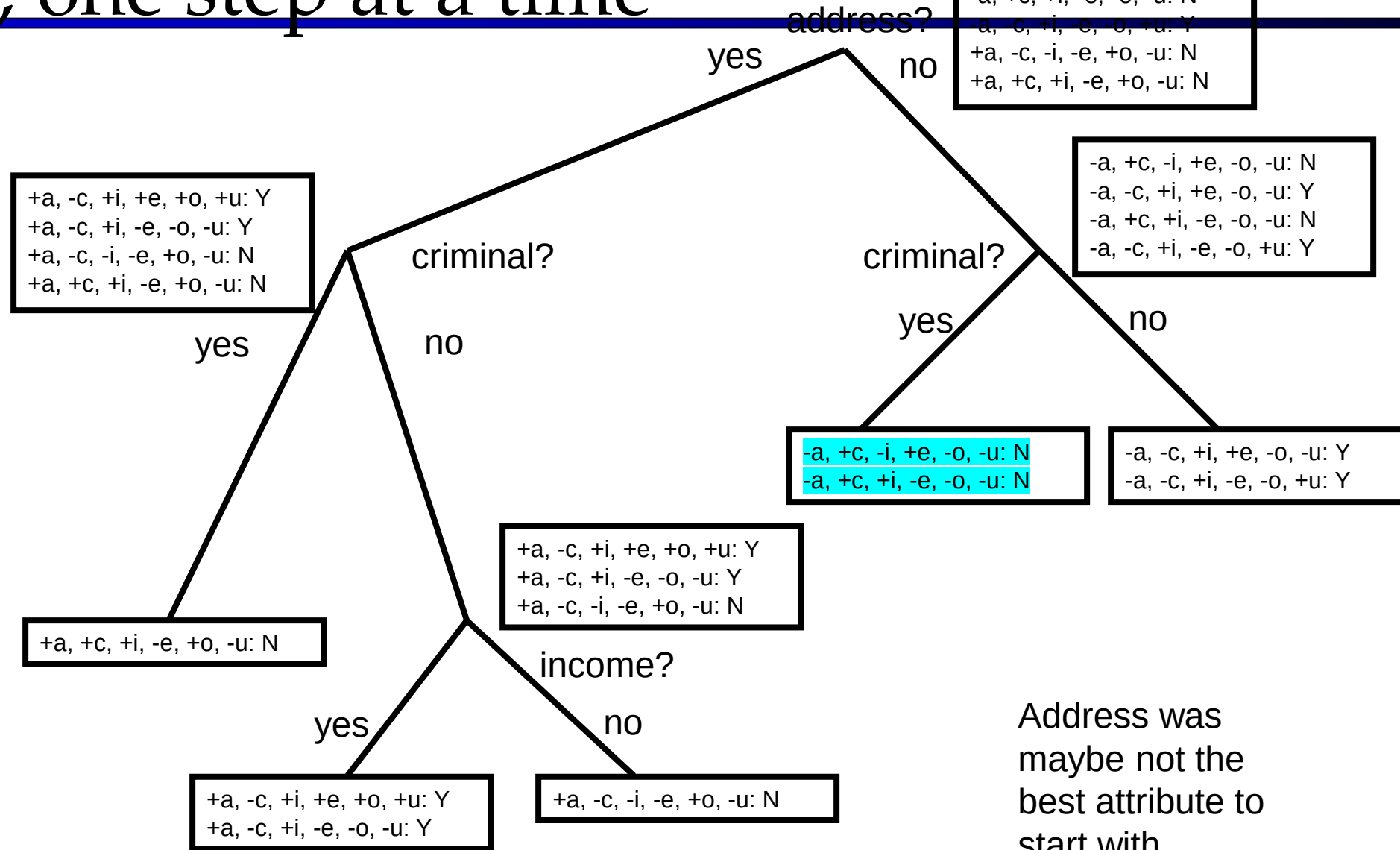
Decision trees



Constructing a decision tree, one step at a time

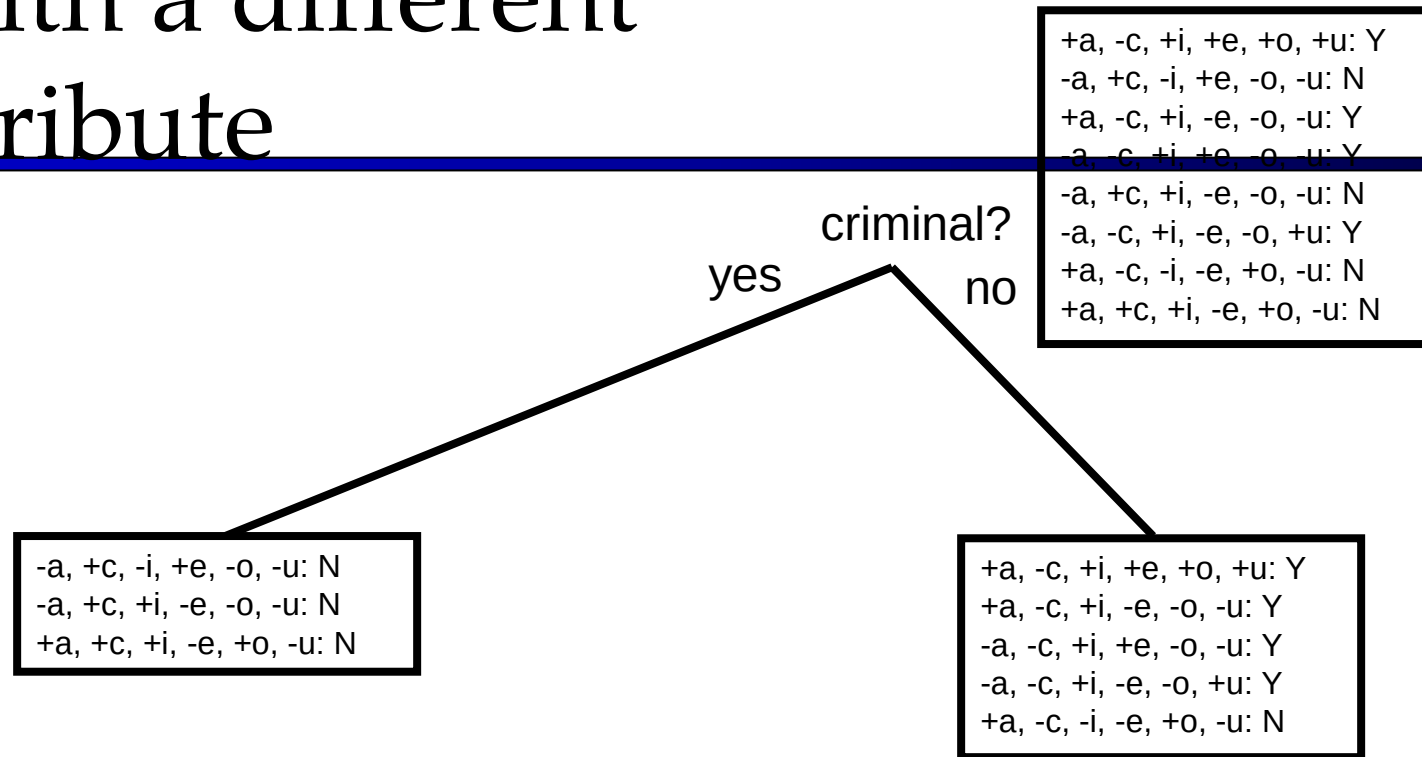


Constructing a decision tree, one step at a time



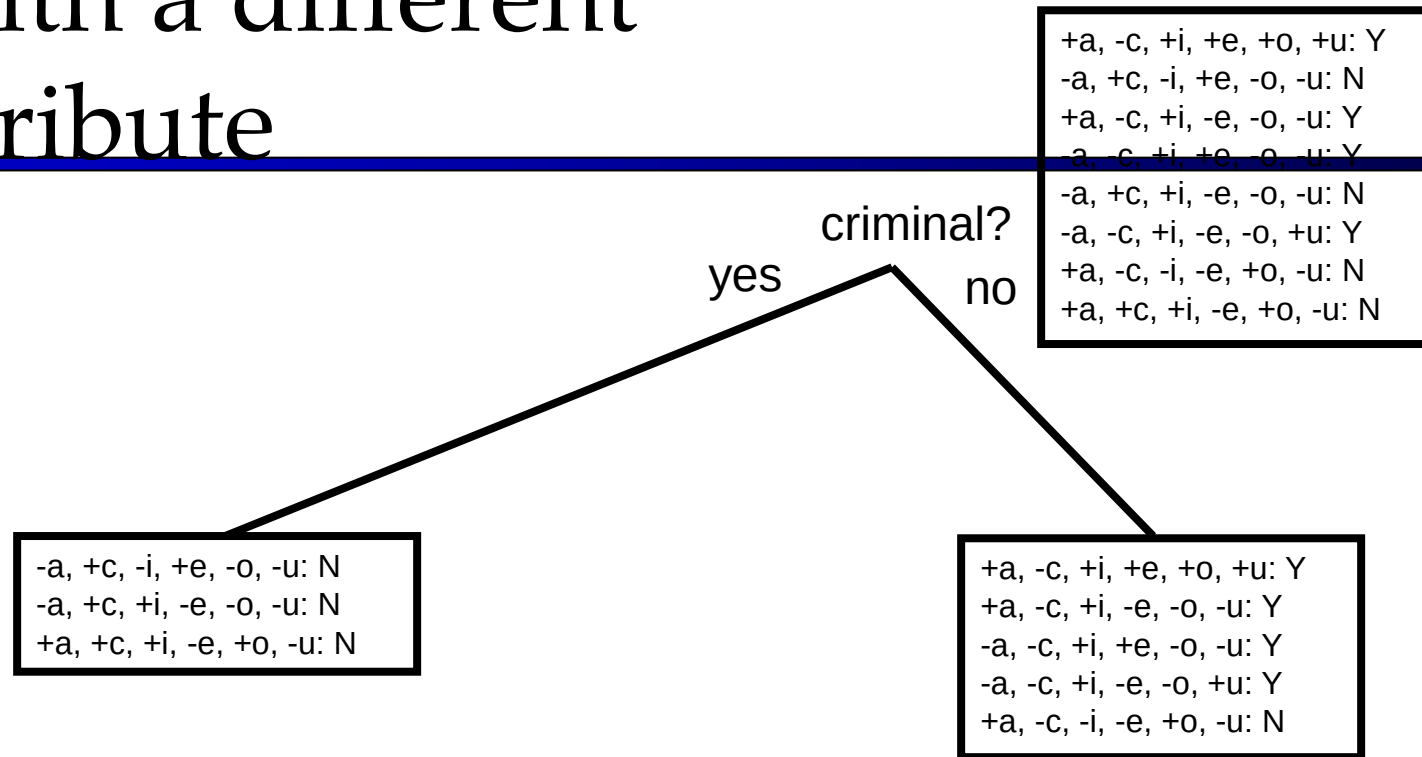
Next person is -a, +c, -i, +e, -o, +u. Will we get paid back? **N**

Starting with a different attribute



- Seems like a much better starting point than address
 - Each node almost completely uniform
 - Almost completely predicts whether we will be paid back

Starting with a different attribute



- Seems like a much better starting point than address
 - Each node almost completely uniform
 - Almost completely predicts whether we will be paid back

Next person is -a, +c, -i, +e, -o, +u. Will we get paid back? **N**

Different approach: nearest neighbor(s)

- Next person is -a, +c, -i, +e, -o, +u. Will we get paid back?
- Nearest neighbor: simply look at most similar example in the training data, see what happened there
 - +a, -c, +i, +e, +o, +u: Y (*distance 4*)
 - a, +c, -i, +e, -o, -u: N (*distance 1*)
 - +a, -c, +i, -e, -o, -u: Y (*distance 5*)
 - a, -c, +i, +e, -o, -u: Y (*distance 3*)
 - a, +c, +i, -e, -o, -u: N (*distance 3*)
 - a, -c, +i, -e, -o, +u: Y (*distance 3*)
 - +a, -c, -i, -e, +o, -u: N (*distance 5*)
 - +a, +c, +i, -e, +o, -u: N (*distance 5*)
- Nearest neighbor is second, so predict N
- k nearest neighbors: look at k nearest neighbors, take a vote
 - E.g., 5 nearest neighbors have 3 Ys, 2Ns, so predict Y

Different approach: nearest neighbor(s)

- Next person is -a, +c, -i, +e, -o, +u. Will we get paid back?
- Nearest neighbor: simply look at most similar example in the training data, see what happened there
 - +a, -c, +i, +e, +o, +u: Y (*distance 4*)
 - a, +c, -i, +e, -o, -u: N (*distance 1*)
 - +a, -c, +i, -e, -o, -u: Y (*distance 5*)
 - a, -c, +i, +e, -o, -u: Y (*distance 3*)
 - a, +c, +i, -e, -o, -u: N (*distance 3*)
 - a, -c, +i, -e, -o, +u: Y (*distance 3*)
 - +a, -c, -i, -e, +o, -u: N (*distance 5*)
 - +a, +c, +i, -e, +o, -u: N (*distance 5*)
- Nearest neighbor is second, so predict N
- k nearest neighbors: look at k nearest neighbors, take a vote
 - E.g., 5 nearest neighbors have 3 Ys, 2Ns, so predict Y

Different approach: nearest neighbor(s)

- Next person is -a, +c, -i, +e, -o, +u. Will we get paid back?
- Nearest neighbor: simply look at most similar example in the training data, see what happened there
 - +a, -c, +i, +e, +o, +u: Y (*distance 4*)
 - a, +c, -i, +e, -o, -u: N (*distance 1*)
 - +a, -c, +i, -e, -o, -u: Y (*distance 5*)
 - a, -c, +i, +e, -o, -u: Y (*distance 3*)
 - a, +c, +i, -e, -o, -u: N (*distance 3*)
 - a, -c, +i, -e, -o, +u: Y (*distance 3*)
 - +a, -c, -i, -e, +o, -u: N (*distance 5*)
 - +a, +c, +i, -e, +o, -u: N (*distance 5*)
- Nearest neighbor is second, so predict N
- k nearest neighbors: look at k nearest neighbors, take a vote
 - E.g., 5 nearest neighbors have 3 Ys, 2Ns, so predict Y

Another approach: **perceptrons**

- Place a weight on every attribute, indicating how important that attribute is (and in which direction it affects things)
- E.g., $w_a = 1$, $w_c = -5$, $w_i = 4$, $w_e = 1$, $w_o = 0$, $w_u = -1$
 - +a, -c, +i, +e, +o, +u: Y (score $1+4+1+0-1 = 5$)
 - a, +c, -i, +e, -o, -u: N (score $-5+1=-4$)
 - +a, -c, +i, -e, -o, -u: Y (score $1+4=5$)
 - a, -c, +i, +e, -o, -u: Y (score $4+1=5$)
 - a, +c, +i, -e, -o, -u: N (score $-5+4=-1$)
 - a, -c, +i, -e, -o, +u: Y (score $4-1=3$)
 - +a, -c, -i, -e, +o, -u: N (score $1+0=1$)
 - +a, +c, +i, -e, +o, -u: N (score $1-5+4+0=0$)
- Need to set some threshold above which we predict to be paid back (say, 2)
- May care about combinations of things (nonlinearity) – generalization: **neural networks**

Another approach: **perceptrons**

- Place a weight on every attribute, indicating how important that attribute is (and in which direction it affects things)
- E.g., $w_a = 1$, $w_c = -5$, $w_i = 4$, $w_e = 1$, $w_o = 0$, $w_u = -1$
 - +a, -c, +i, +e, +o, +u: Y (score $1+4+1+0-1 = 5$)
 - a, +c, -i, +e, -o, -u: N (score $-5+1=-4$)
 - +a, -c, +i, -e, -o, -u: Y (score $1+4=5$)
 - a, -c, +i, +e, -o, -u: Y (score $4+1=5$)
 - a, +c, +i, -e, -o, -u: N (score $-5+4=-1$)
 - a, -c, +i, -e, -o, +u: Y (score $4-1=3$)
 - +a, -c, -i, -e, +o, -u: N (score $1+0=1$)
 - +a, +c, +i, -e, +o, -u: N (score $1-5+4+0=0$)
- Need to set some threshold above which we predict to be paid back (say, 2)
- May care about combinations of things (nonlinearity) – generalization: **neural networks**

Another approach: **perceptrons**

- Place a weight on every attribute, indicating how important that attribute is (and in which direction it affects things)
- E.g., $w_a = 1$, $w_c = -5$, $w_i = 4$, $w_e = 1$, $w_o = 0$, $w_u = -1$
 - +a, -c, +i, +e, +o, +u: Y (score $1+4+1+0-1 = 5$)
 - a, +c, -i, +e, -o, -u: N (score $-5+1=-4$)
 - +a, -c, +i, -e, -o, -u: Y (score $1+4=5$)
 - a, -c, +i, +e, -o, -u: Y (score $4+1=5$)
 - a, +c, +i, -e, -o, -u: N (score $-5+4=-1$)
 - a, -c, +i, -e, -o, +u: Y (score $4-1=3$)
 - +a, -c, -i, -e, +o, -u: N (score $1+0=1$)
 - +a, +c, +i, -e, +o, -u: N (score $1-5+4+0=0$)
- Need to set some threshold above which we predict to be paid back (say, 2)
- May care about combinations of things (nonlinearity) – generalization: **neural networks**

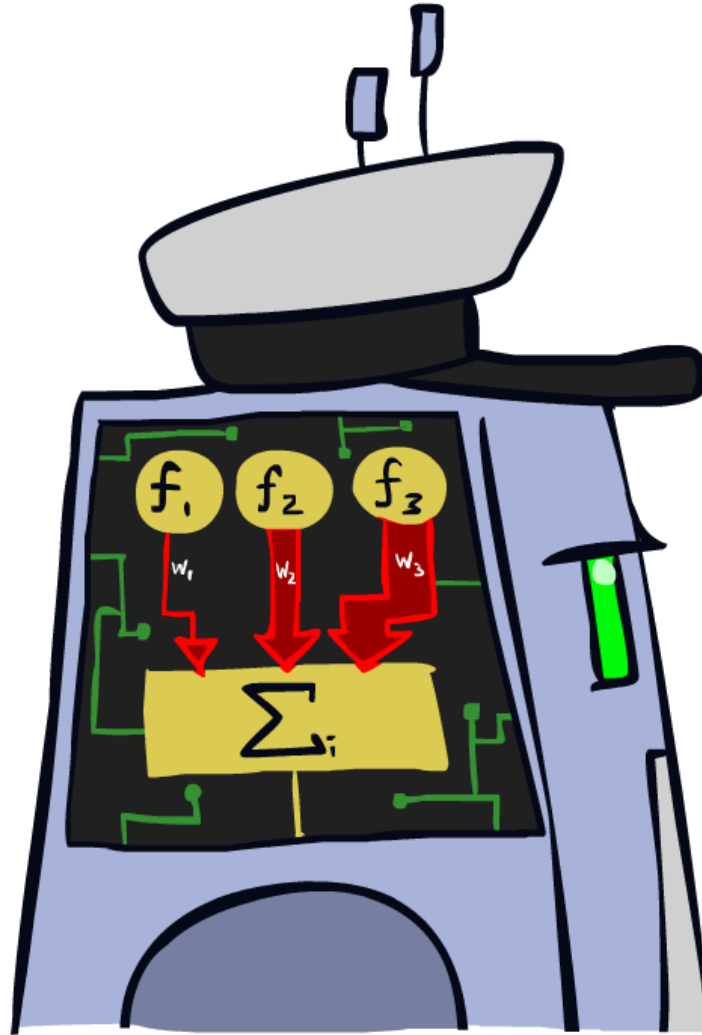
Next person is -a, +c, -i, +e, -o, +u. Will we get paid back?

Score $-1-5-4+1-0-1=-10$, Prediction: N

Supervised Learning

- Linear Models
- Nonlinear Models

Linear Classifiers



Feature Vectors

 x

Hello,

Do you want free
printr cartridges?
Why pay more when
you can get them
ABSOLUTELY FREE!
Just

2

 $f(x)$

free : 2
YOUR_NAME : 0
MISSPELLED : 2
FROM_FRIEND : 0
...

PIXEL-7,12 : 1
PIXEL-7,13 : 0
...
NUM_LOOPS : 1
...

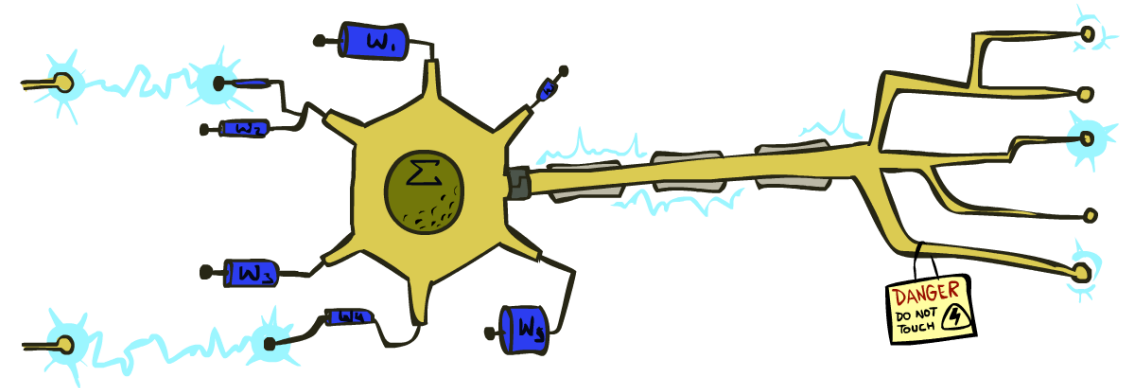
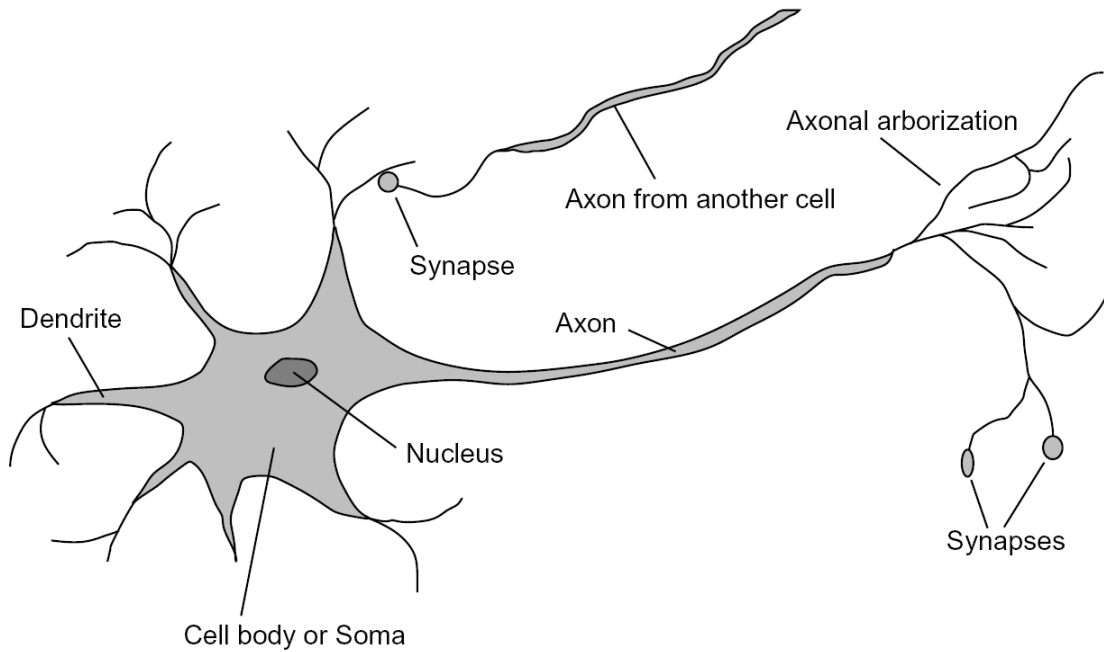
 y

SPAM
or
+

"2"

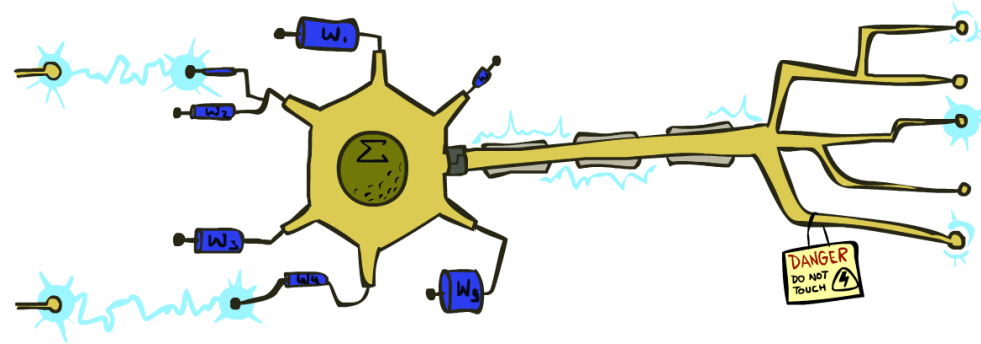
Some (Simplified) Biology

- Very loose inspiration: human neurons



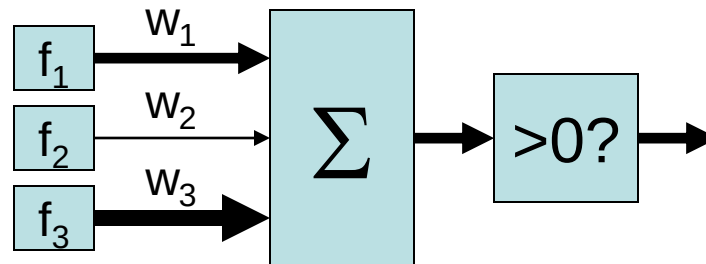
Linear Classifiers

- Inputs are **feature values**
- Each feature has a **weight**
- Sum is the **activation**



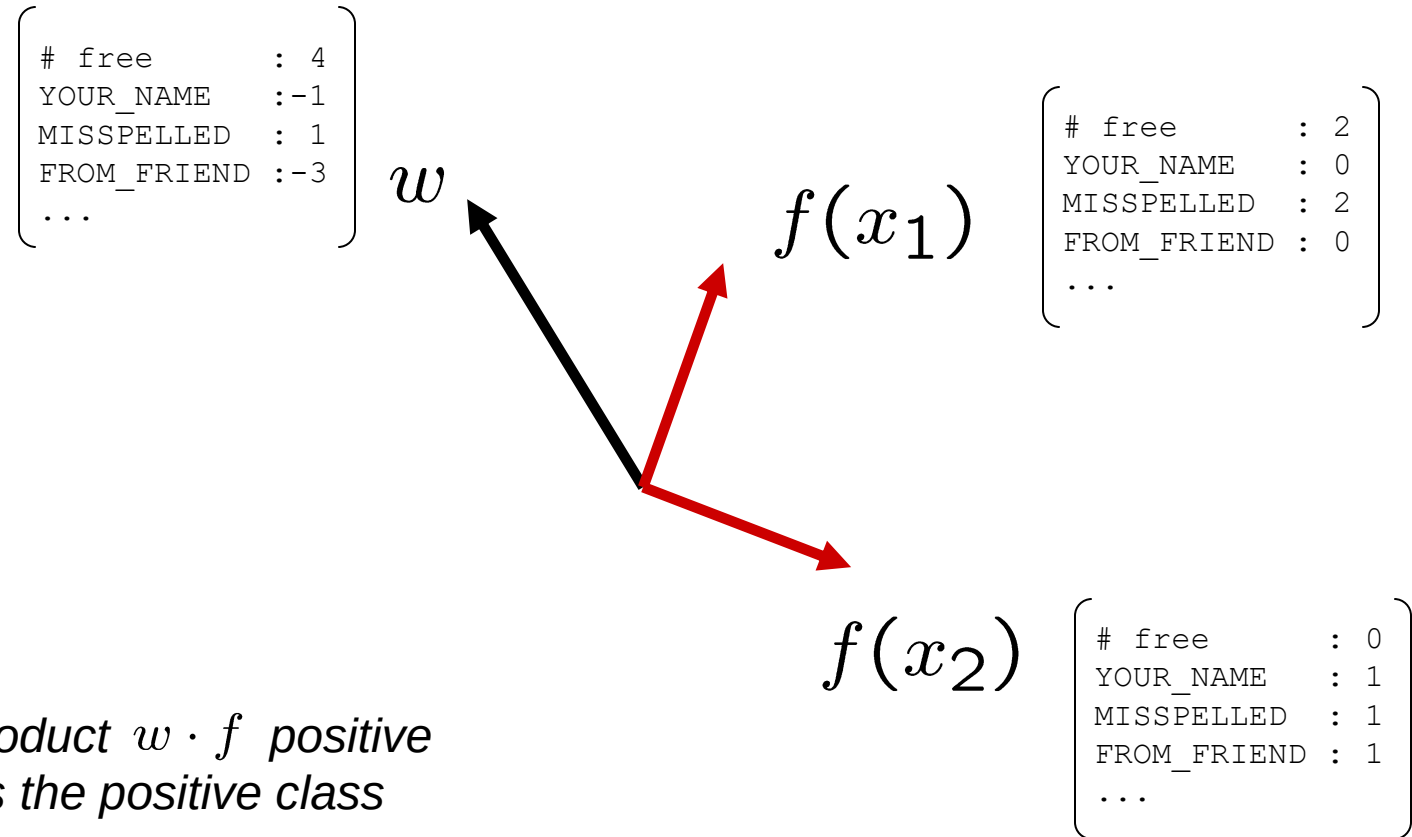
$$\text{activation}_w(x) = \sum_i w_i \cdot f_i(x) = w \cdot f(x)$$

- If the activation is:
 - Positive, output +1
 - Negative, output -1

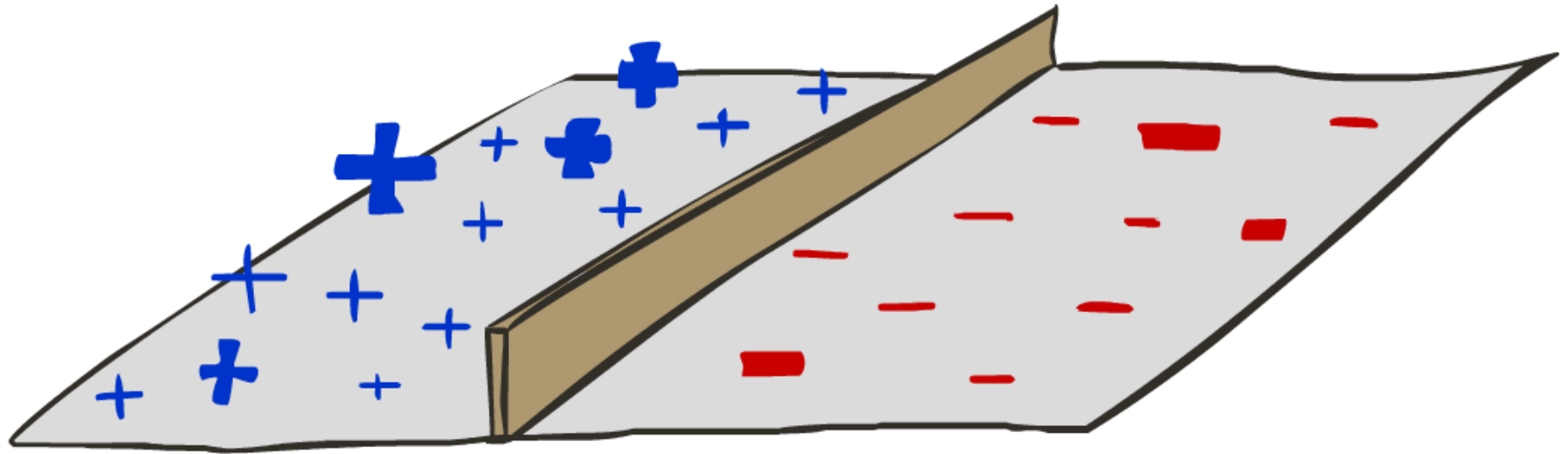


Weights

- Binary case: compare features to a weight vector
- Learning: figure out the weight vector from examples



Decision Rules

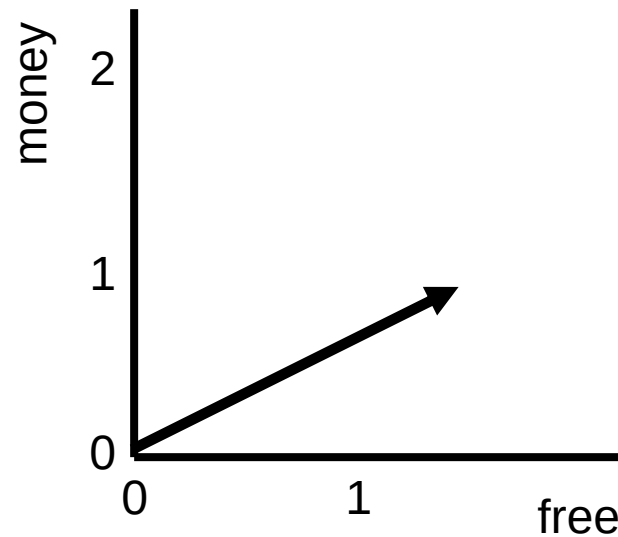
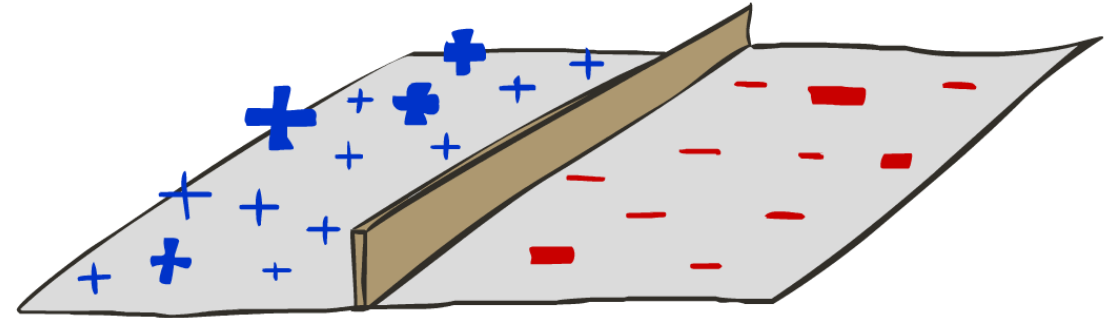


Binary Decision Rule

- In the space of feature vectors
 - Examples are points
 - Any weight vector is a hyperplane
 - One side corresponds to $Y=+1$
 - Other corresponds to $Y=-1$

w

BIAS	:	-3
free	:	4
money	:	2
...		

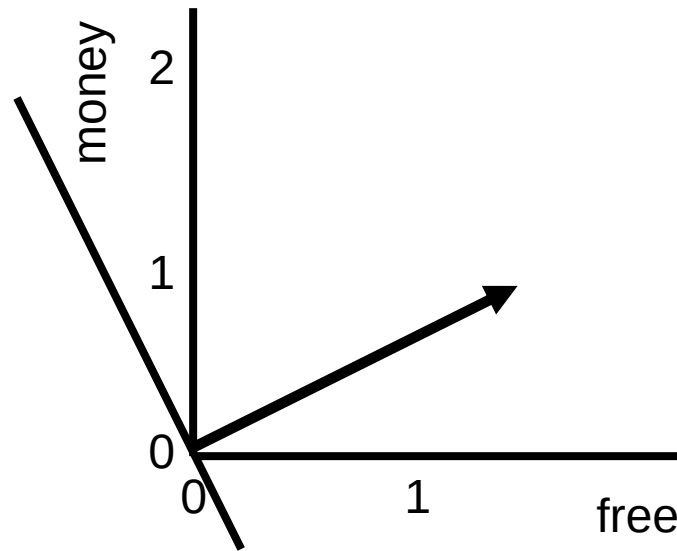
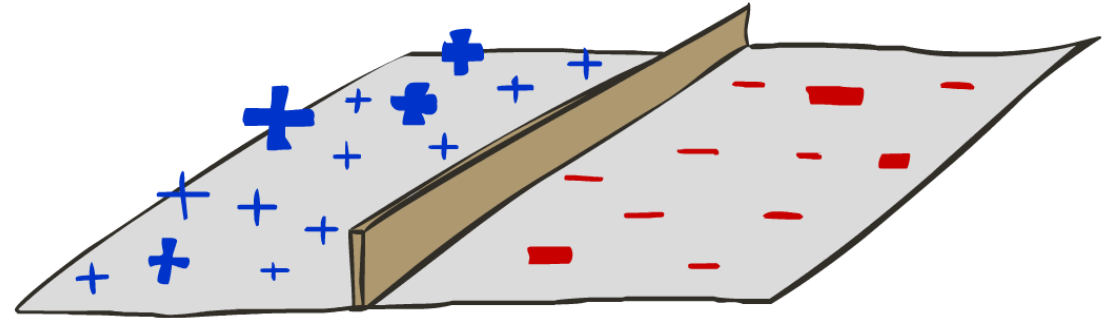


Binary Decision Rule

- In the space of feature vectors
 - Examples are points
 - Any weight vector is a hyperplane
 - One side corresponds to $Y=+1$
 - Other corresponds to $Y=-1$

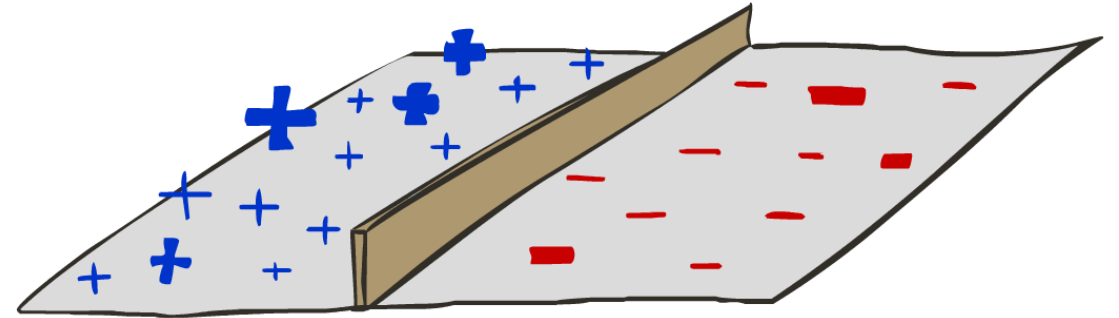
w

BIAS	:	-3
free	:	4
money	:	2
...		



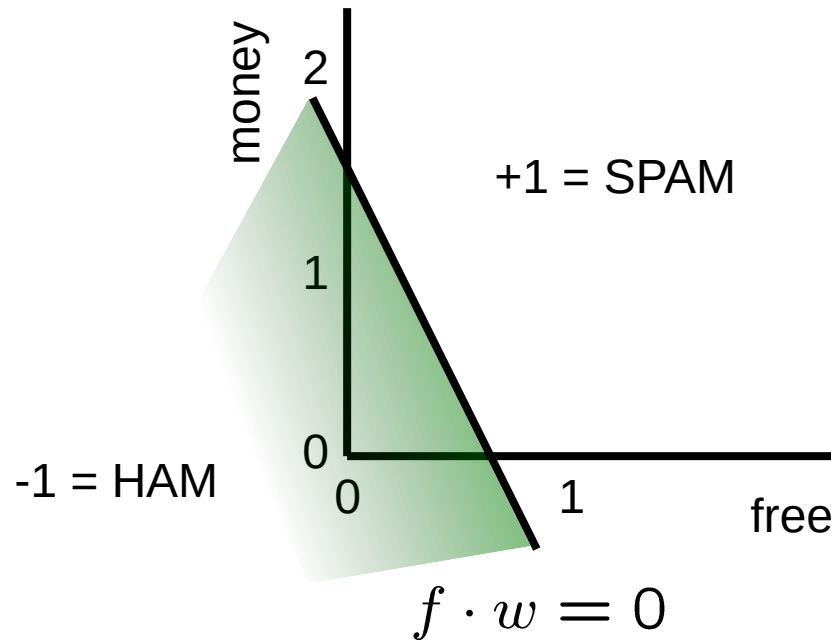
Binary Decision Rule

- In the space of feature vectors
 - Examples are points
 - Any weight vector is a hyperplane
 - One side corresponds to $Y=+1$
 - Other corresponds to $Y=-1$

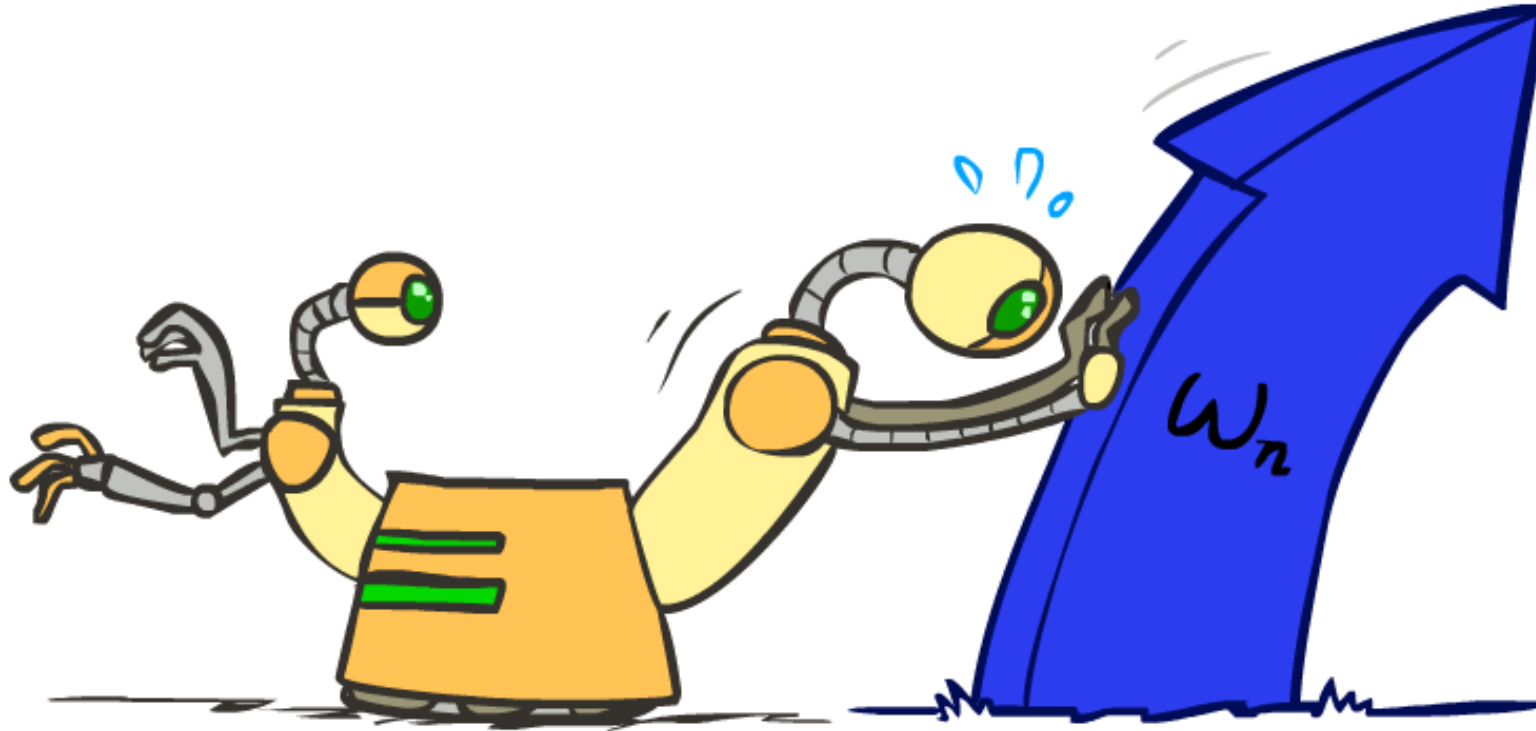


w

BIAS	:	-3
free	:	4
money	:	2
...		

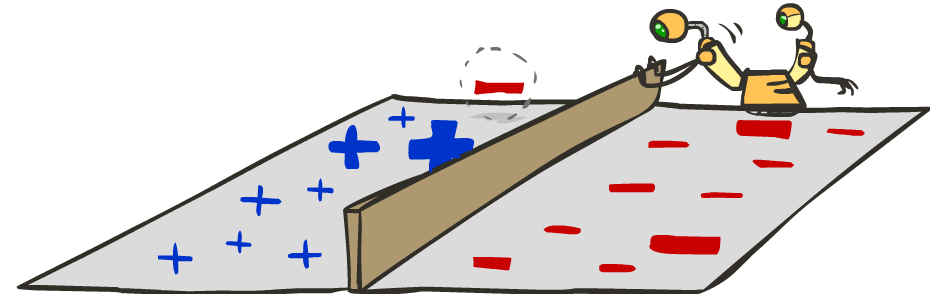
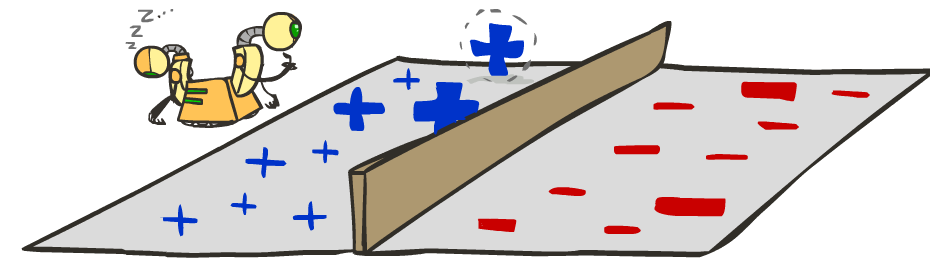
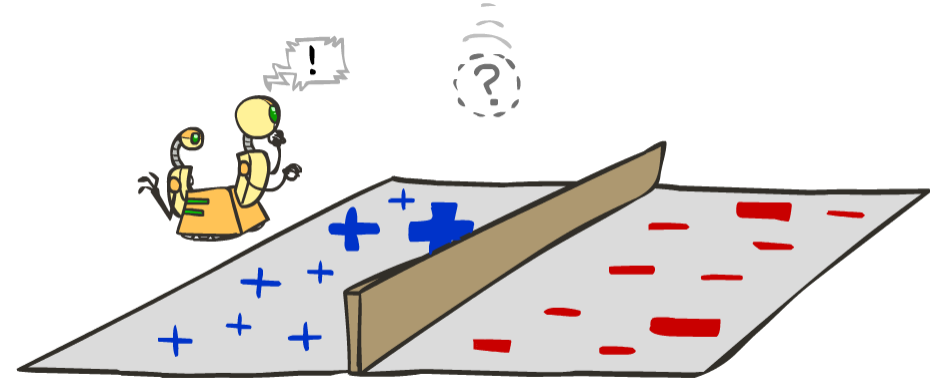


Weight Updates



Learning: Binary Perceptron

- Start with weights = 0
- For each training instance:
 - Classify with current weights
- If correct (i.e., $y=y^*$), no change!
- If wrong: adjust the weight vector



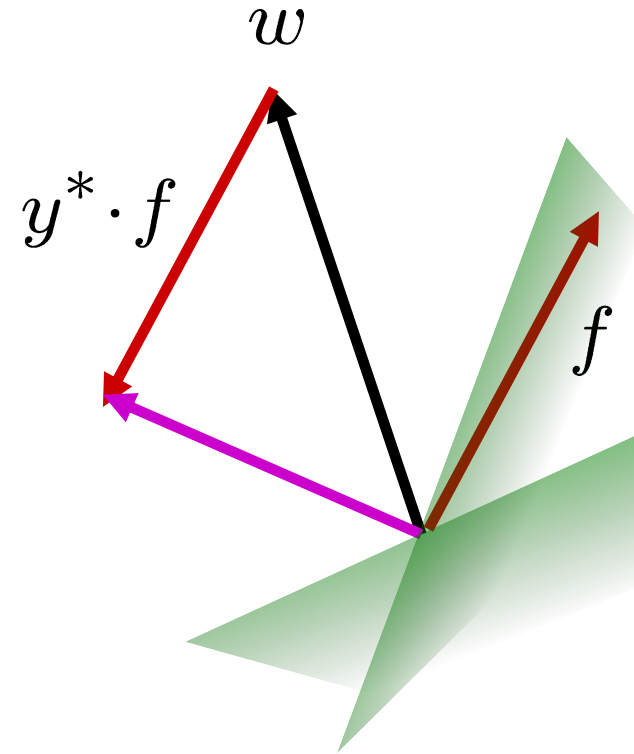
Learning: Binary Perceptron

- Start with weights = 0
- For each training instance:
 - Classify with current weights

$$y = \begin{cases} +1 & \text{if } w \cdot f(x) \geq 0 \\ -1 & \text{if } w \cdot f(x) < 0 \end{cases}$$

- If correct (i.e., $y=y^*$), no change!
- If wrong: adjust the weight vector by adding or subtracting the feature vector. Subtract if y^* is -1.

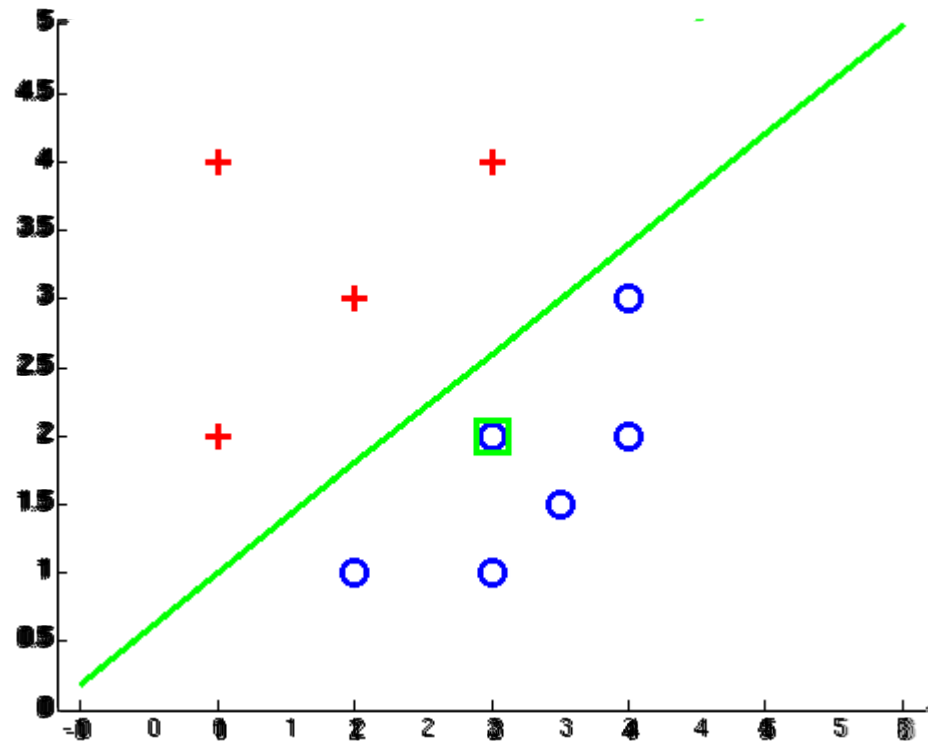
$$w = w + y^* \cdot f$$



Before: $w \cdot f$
After: $w \cdot f + y^* \cdot f$
 $f \cdot f \geq 0$

Examples: Perceptron

- Separable Case



Multiclass Decision Rule

- If we have multiple classes:
 - A weight vector for each class:

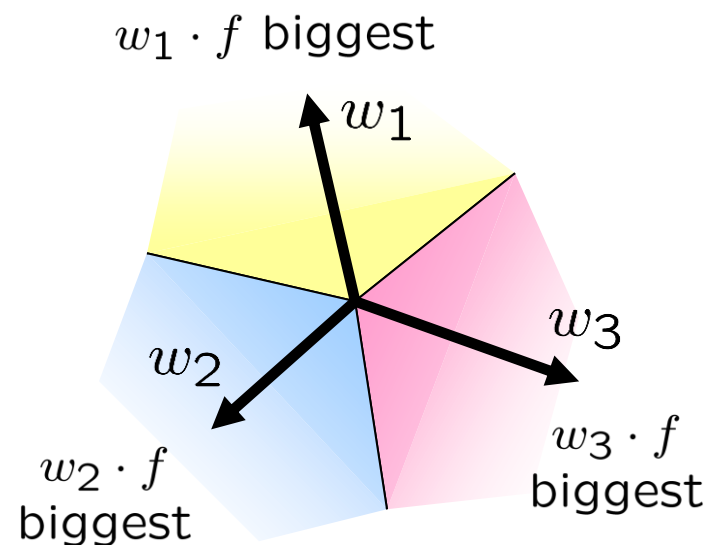
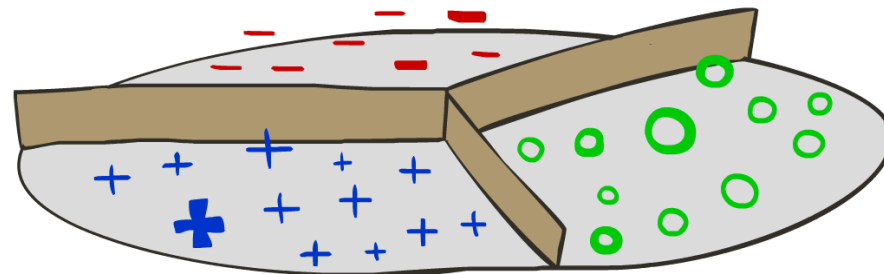
$$w_y$$

- Score (activation) of a class y :

$$w_y \cdot f(x)$$

- Prediction highest score wins

$$y = \arg \max_y w_y \cdot f(x)$$



Binary = multiclass where the negative class has weight zero

Learning: Multiclass Perceptron

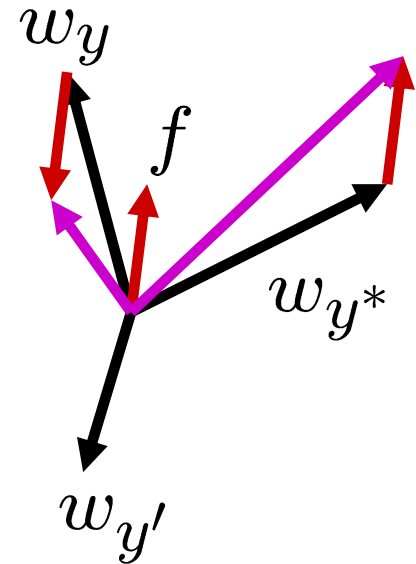
- Start with all weights = 0
- Pick up training examples one by one
- Predict with current weights

$$y = \arg \max_y w_y \cdot f(x)$$

- If correct, no change!
- If wrong: lower score of wrong answer, raise score of right answer

$$w_y = w_y - f(x)$$

$$w_{y^*} = w_{y^*} + f(x)$$

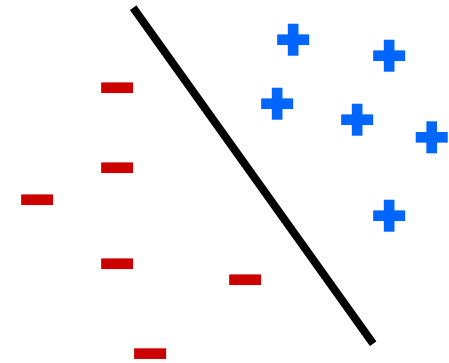


Properties of Perceptrons

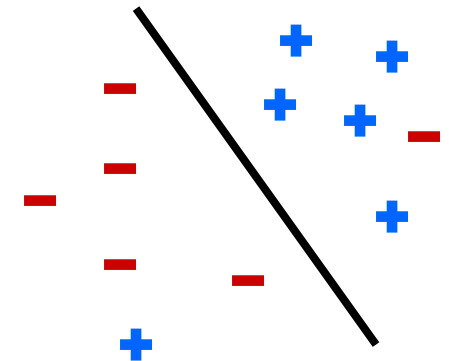
- Separability: true if some parameters get the training set perfectly correct
- Convergence: if the training is separable, perceptron will eventually converge (binary case)
- Mistake Bound: the maximum number of mistakes (binary case) related to the *margin* or degree of separability

$$\text{mistakes} < \frac{k}{\delta^2}$$

Separable

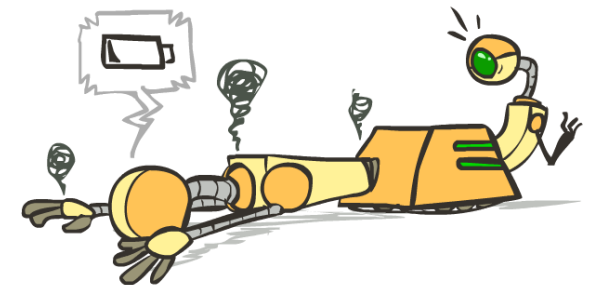
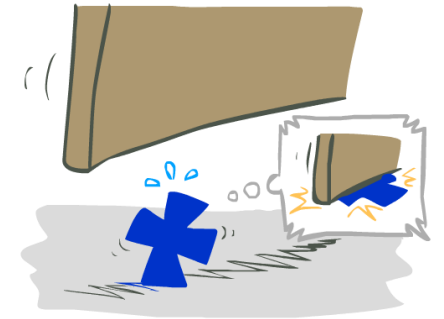
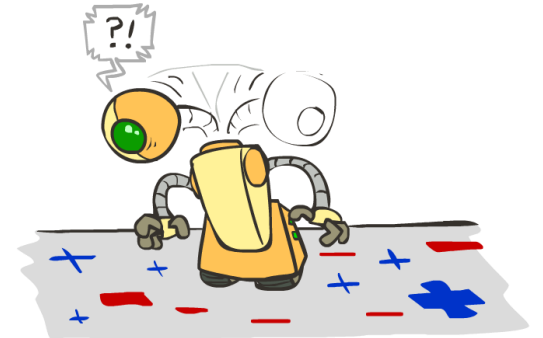
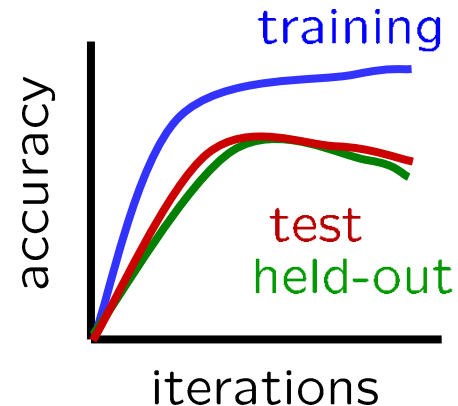
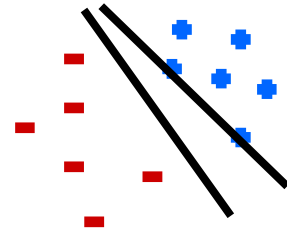
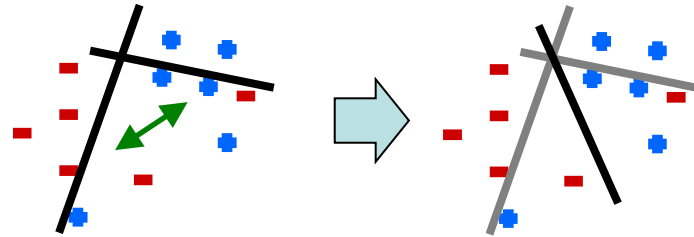


Non-Separable

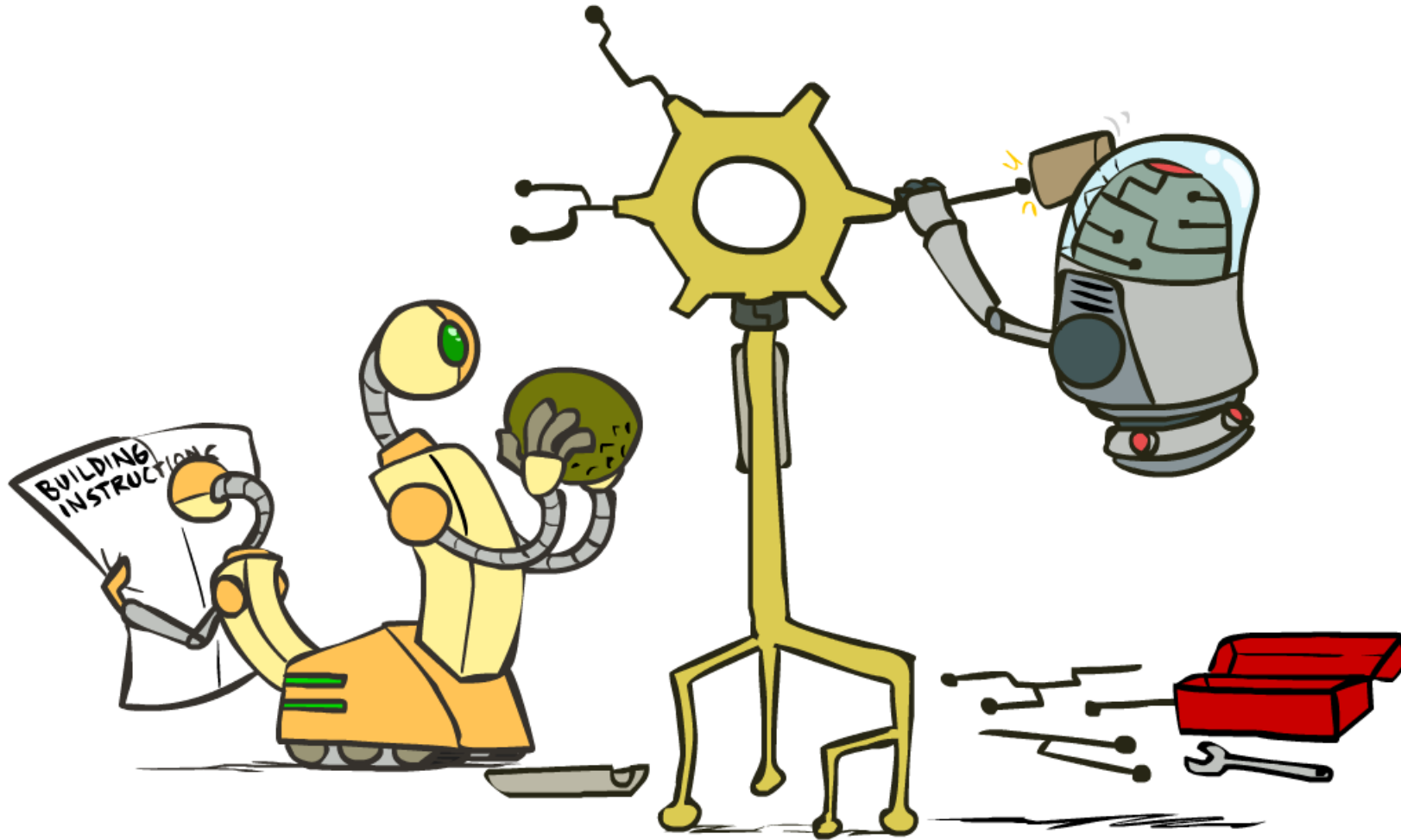


Problems with the Perceptron

- Noise: if the data isn't separable, weights might thrash
 - Averaging weight vectors over time can help (averaged perceptron)
- Mediocre generalization: finds a "barely" separating solution
- Overtraining: test / held-out accuracy usually rises, then falls
 - Overtraining is a kind of overfitting

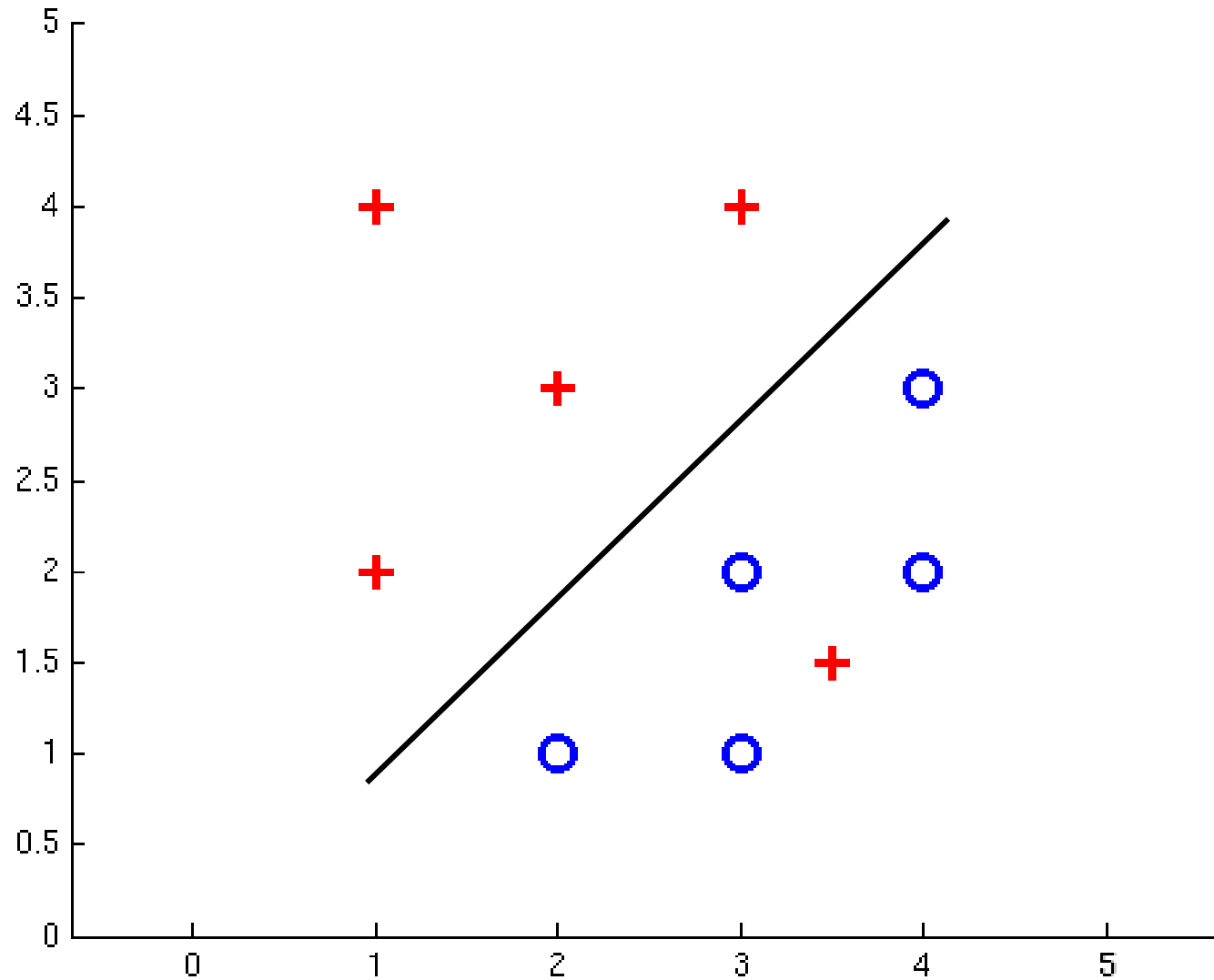


Improving the Perceptron

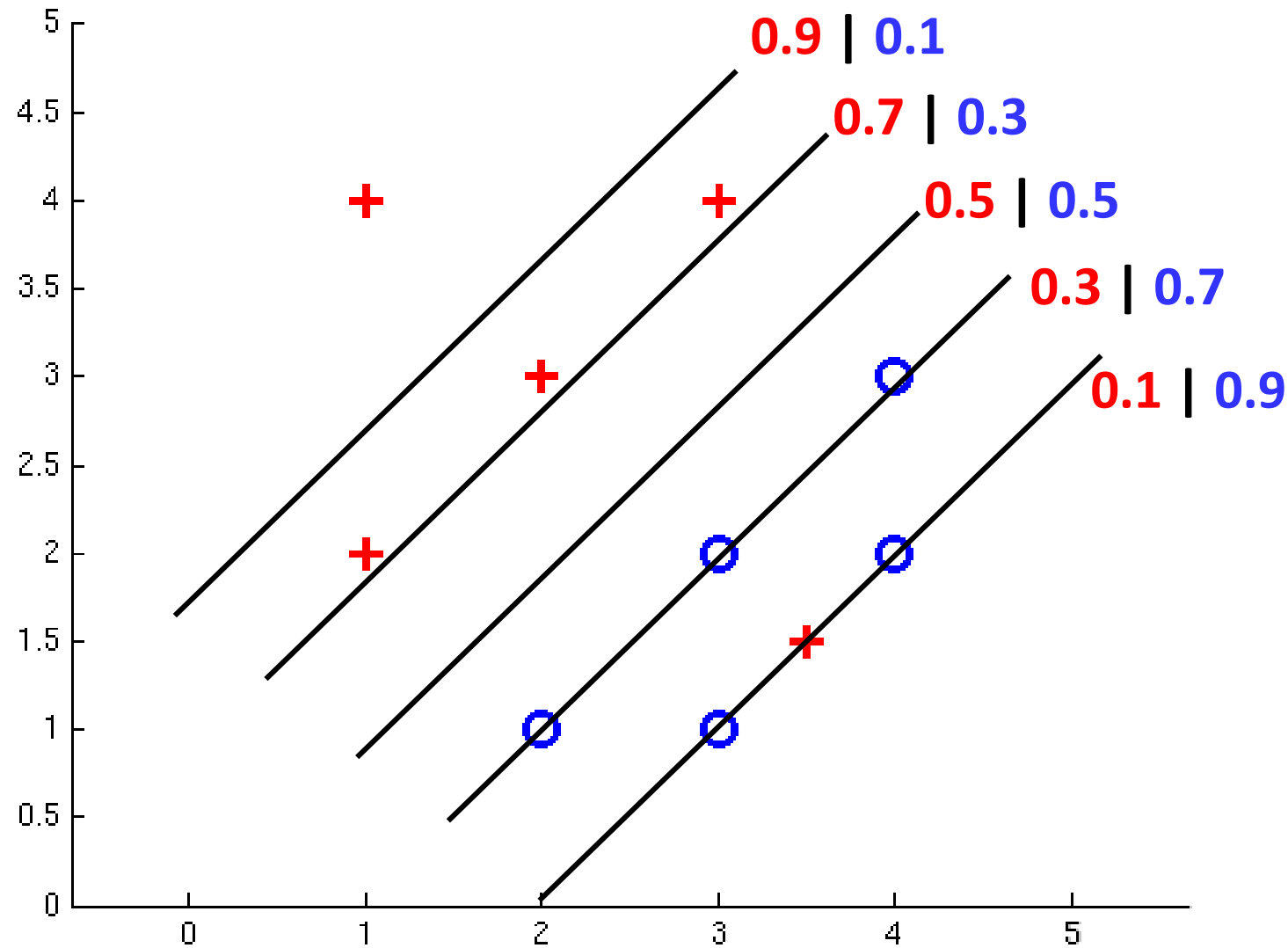


Non-Separable Case: Deterministic Decision

Even the best linear boundary makes at least one mistake



Non-Separable Case: Probabilistic Decision

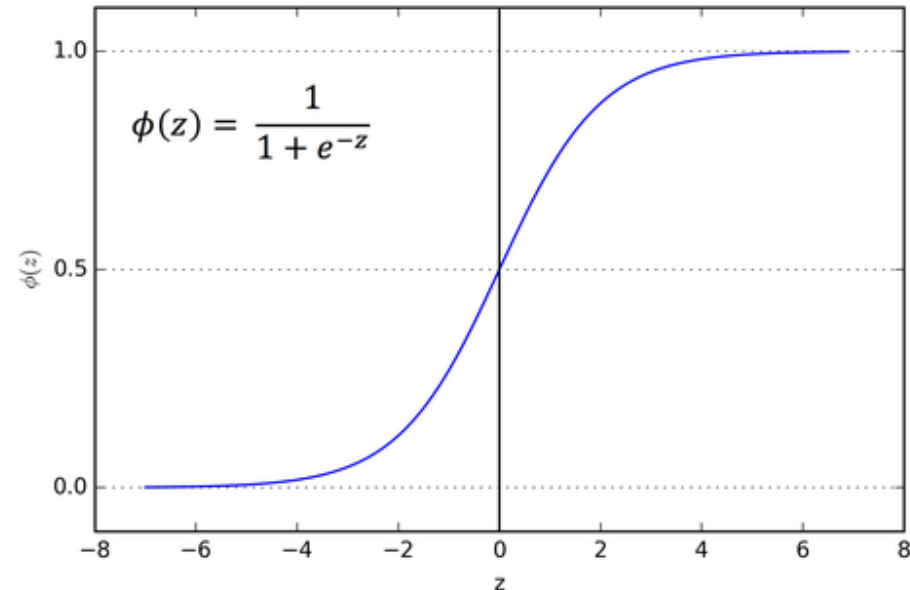


How to get probabilistic decisions?

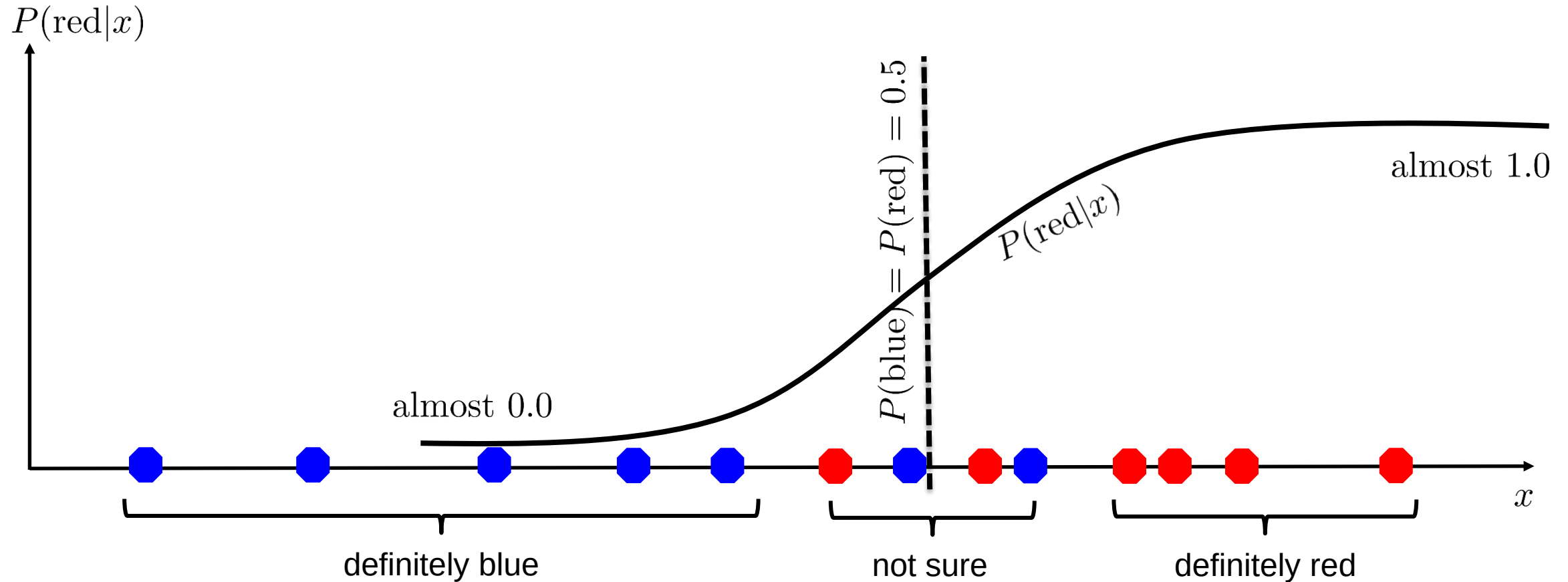
- Perceptron scoring: $z = w \cdot f(x)$
- If $z = w \cdot f(x)$ very positive $\rightarrow$ want probability going to 1
- If $z = w \cdot f(x)$ very negative $\rightarrow$ want probability going to 0

- Sigmoid function

$$\phi(z) = \frac{1}{1 + e^{-z}}$$



A 1D Example

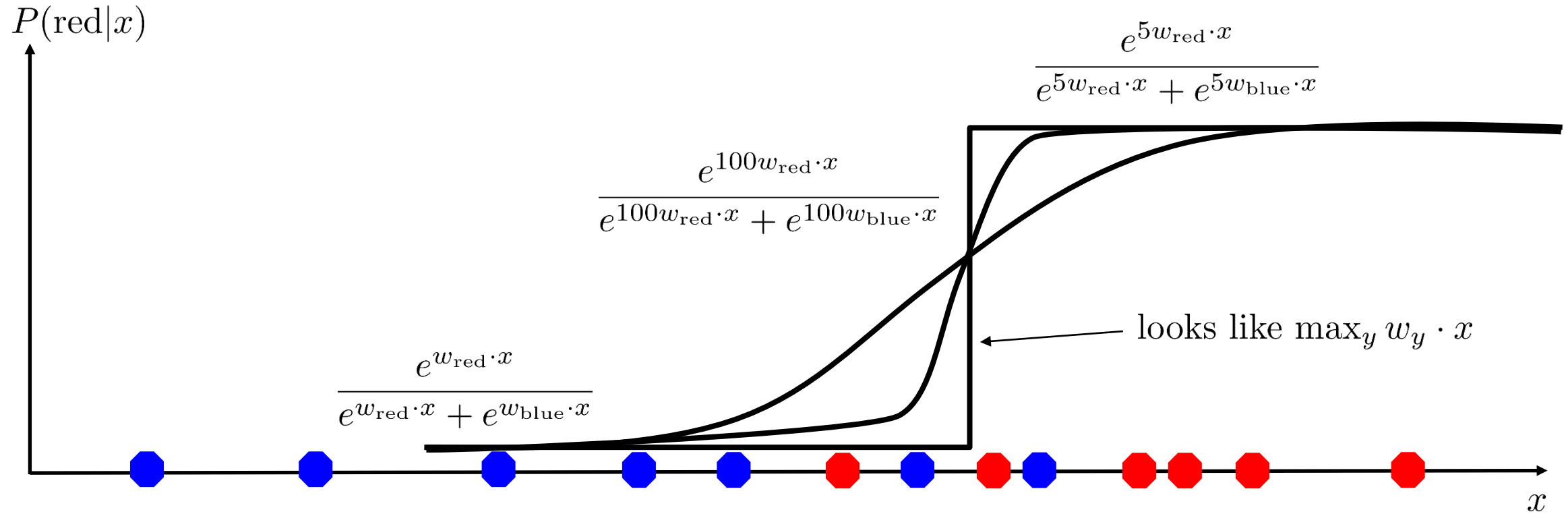


$$P(\text{red}|x) = \frac{e^{w_{\text{red}} \cdot x}}{e^{w_{\text{red}} \cdot x} + e^{w_{\text{blue}} \cdot x}}$$

probability increases exponentially as we move away from boundary

normalizer

The *Soft* Max



$$P(\text{red}|x) = \frac{e^{w_{\text{red}} \cdot x}}{e^{w_{\text{red}} \cdot x} + e^{w_{\text{blue}} \cdot x}}$$

Best w ?

- Maximum likelihood estimation:

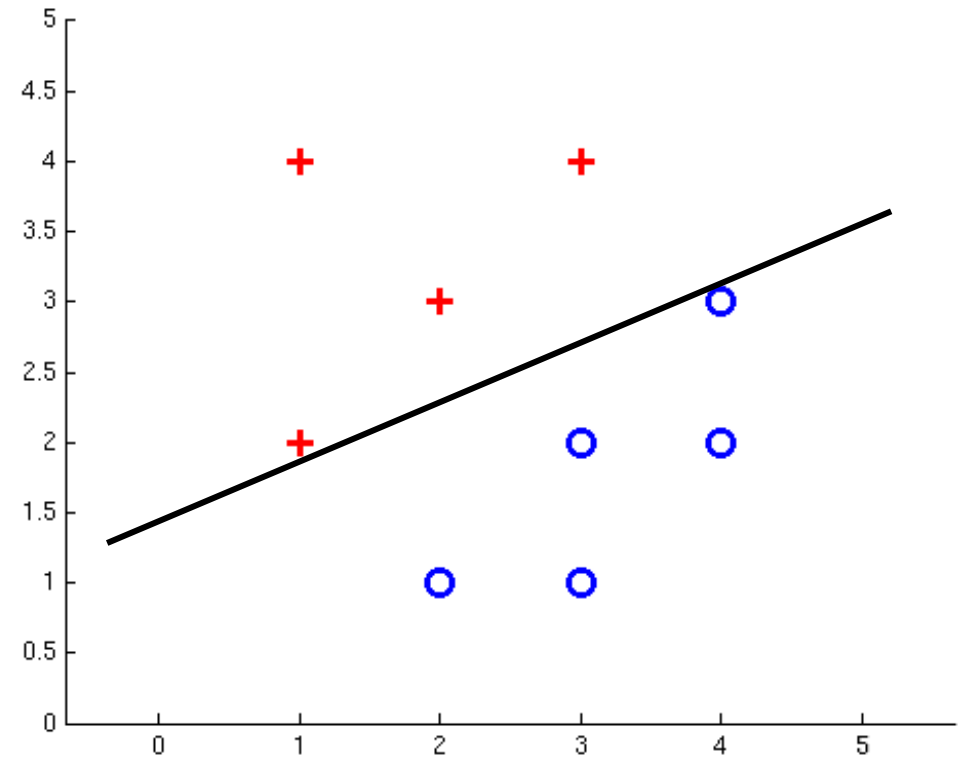
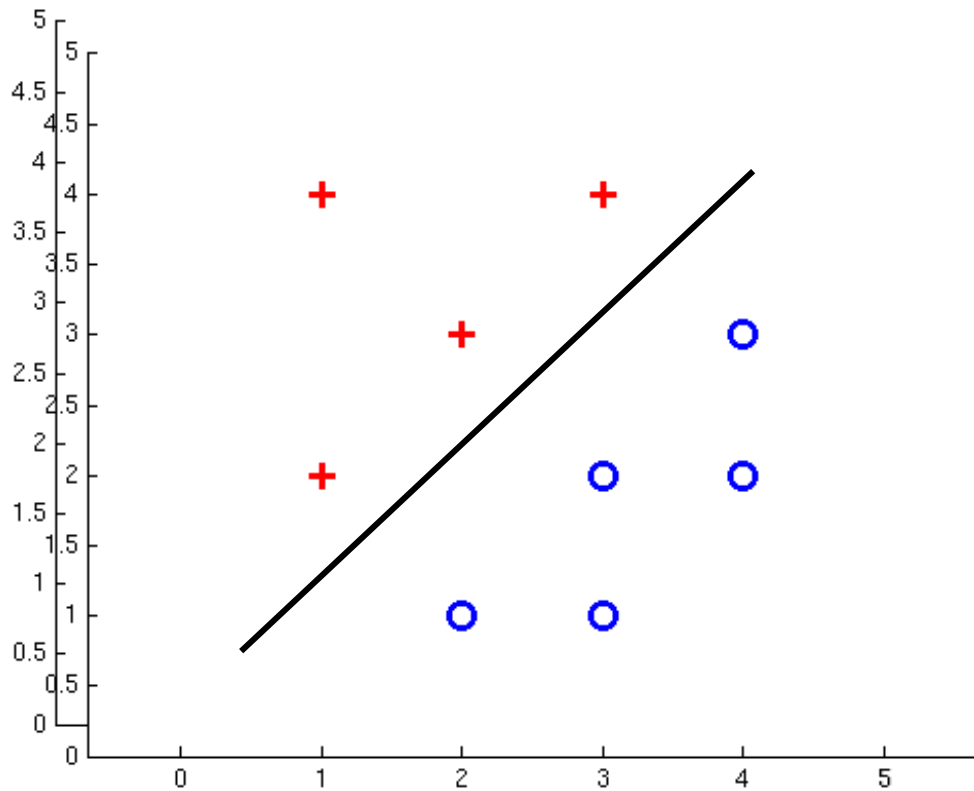
$$\max_w ll(w) = \max_w \sum_i \log P(y^{(i)} | x^{(i)}; w)$$

with:

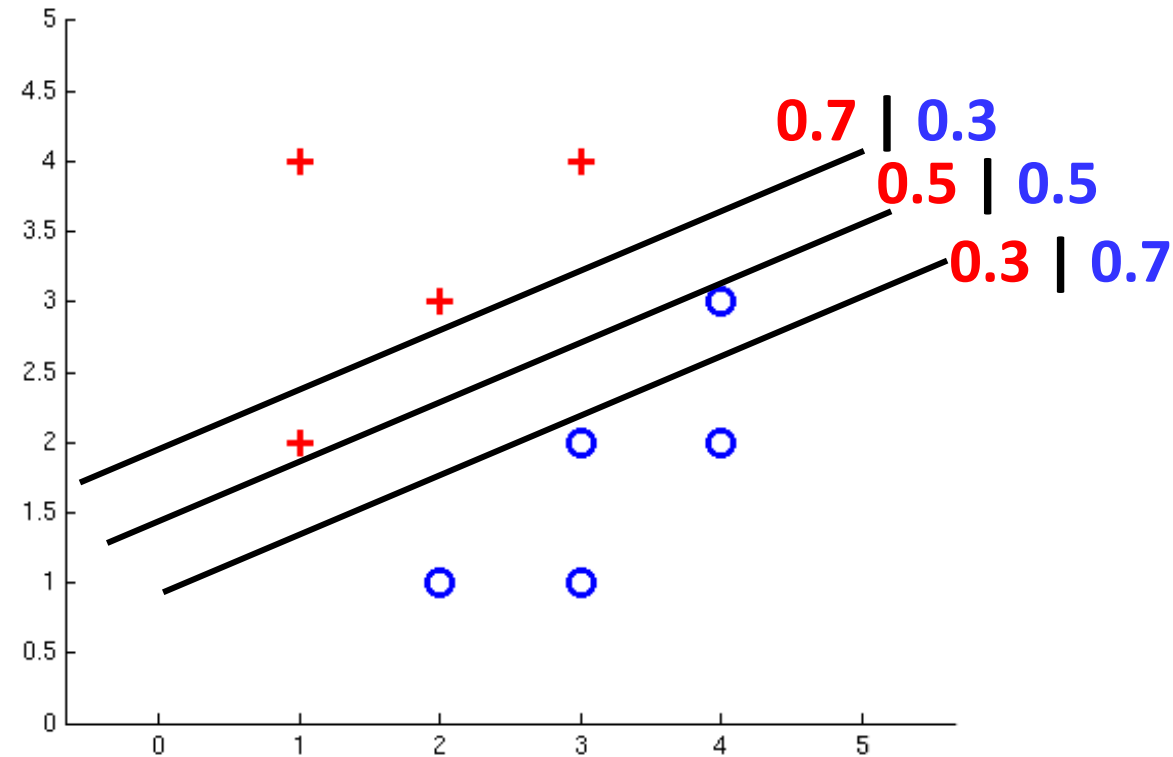
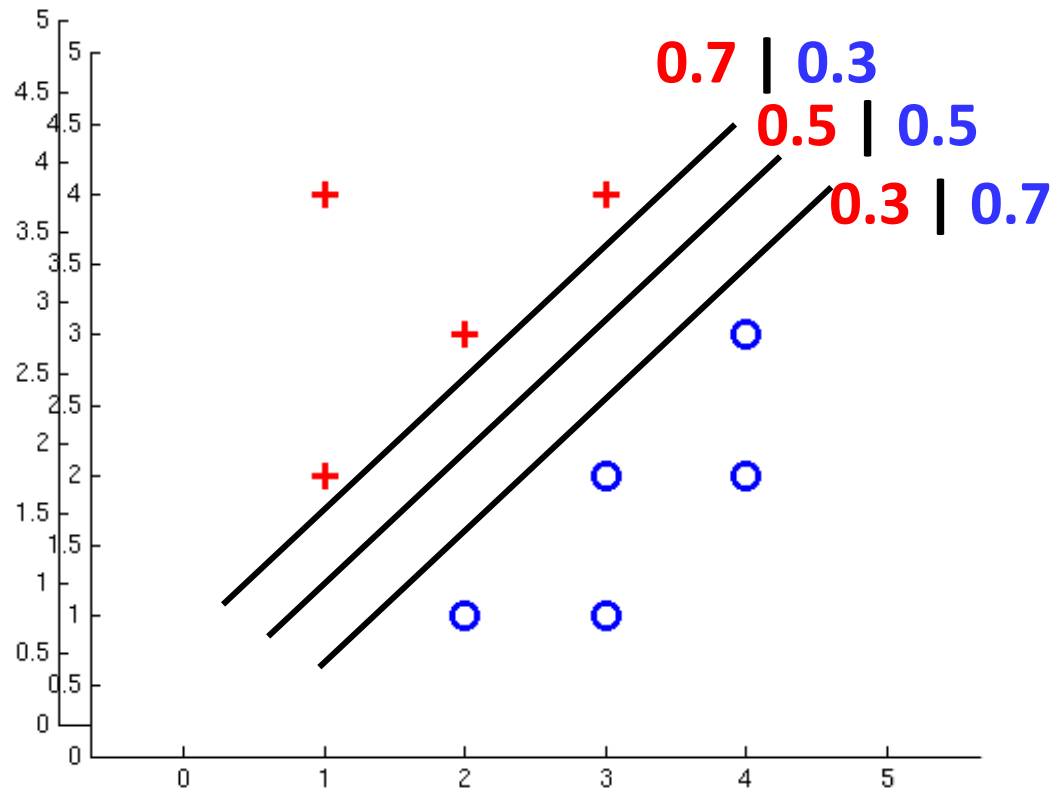
$$P(y^{(i)} = +1 | x^{(i)}; w) = \frac{1}{1 + e^{-w \cdot f(x^{(i)})}}$$
$$P(y^{(i)} = -1 | x^{(i)}; w) = 1 - \frac{1}{1 + e^{-w \cdot f(x^{(i)})}}$$

= Logistic Regression

Separable Case: Deterministic Decision – Many Options



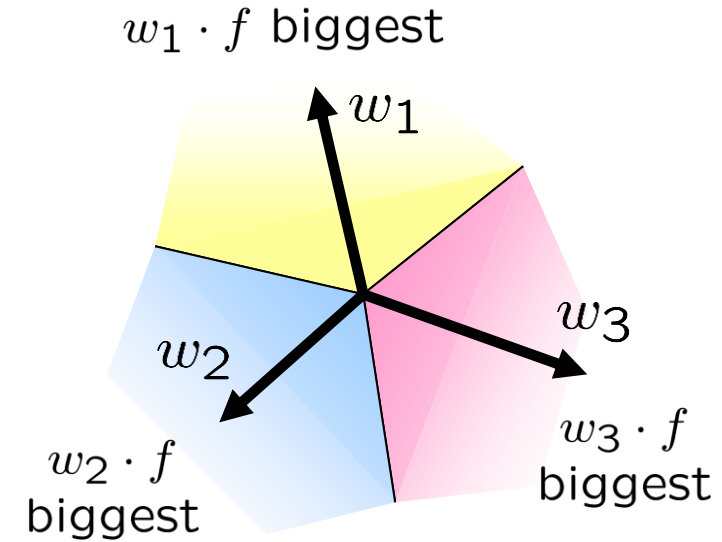
Separable Case: Probabilistic Decision – Clear Preference



Multiclass Logistic Regression

- Recall Perceptron:

- A weight vector for each class: w_y
- Score (activation) of a class y : $w_y \cdot f(x)$
- Prediction highest score wins $y = \arg \max_y w_y \cdot f(x)$



- How to make the scores into probabilities?

$$\underbrace{z_1, z_2, z_3}_{\text{original activations}} \rightarrow \underbrace{\frac{e^{z_1}}{e^{z_1} + e^{z_2} + e^{z_3}}, \frac{e^{z_2}}{e^{z_1} + e^{z_2} + e^{z_3}}, \frac{e^{z_3}}{e^{z_1} + e^{z_2} + e^{z_3}}}_{\text{softmax activations}}$$

Best w ?

- Maximum likelihood estimation:

$$\max_w ll(w) = \max_w \sum_i \log P(y^{(i)} | x^{(i)}; w)$$

with:

$$P(y^{(i)} | x^{(i)}; w) = \frac{e^{w_{y^{(i)}} \cdot f(x^{(i)})}}{\sum_y e^{w_y \cdot f(x^{(i)})}}$$

= Multi-Class Logistic Regression

Next Lecture

- Optimization

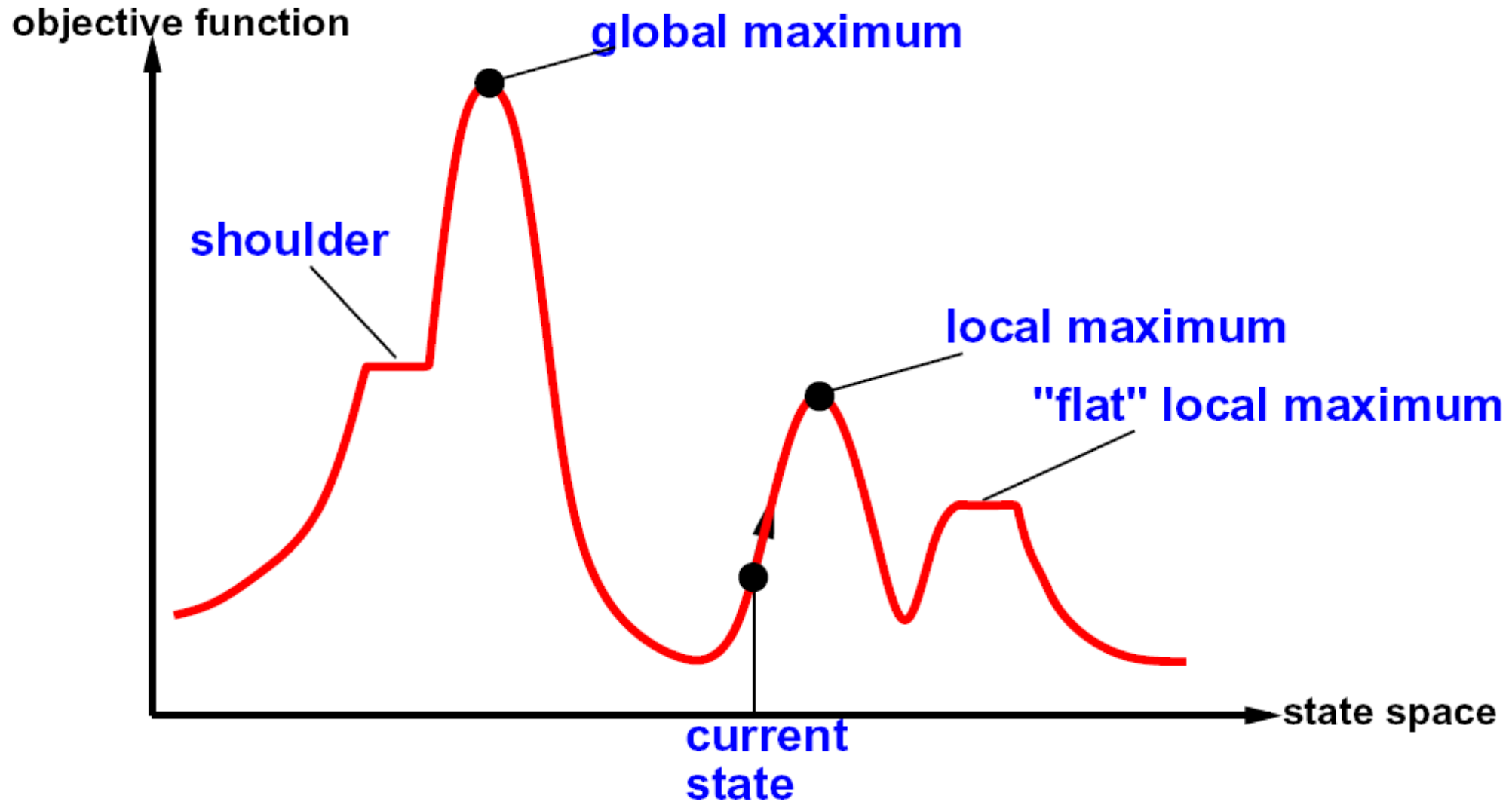
- i.e., how do we solve:

$$\max_w ll(w) = \max_w \sum_i \log P(y^{(i)} | x^{(i)}; w)$$

Local Search & Global Search

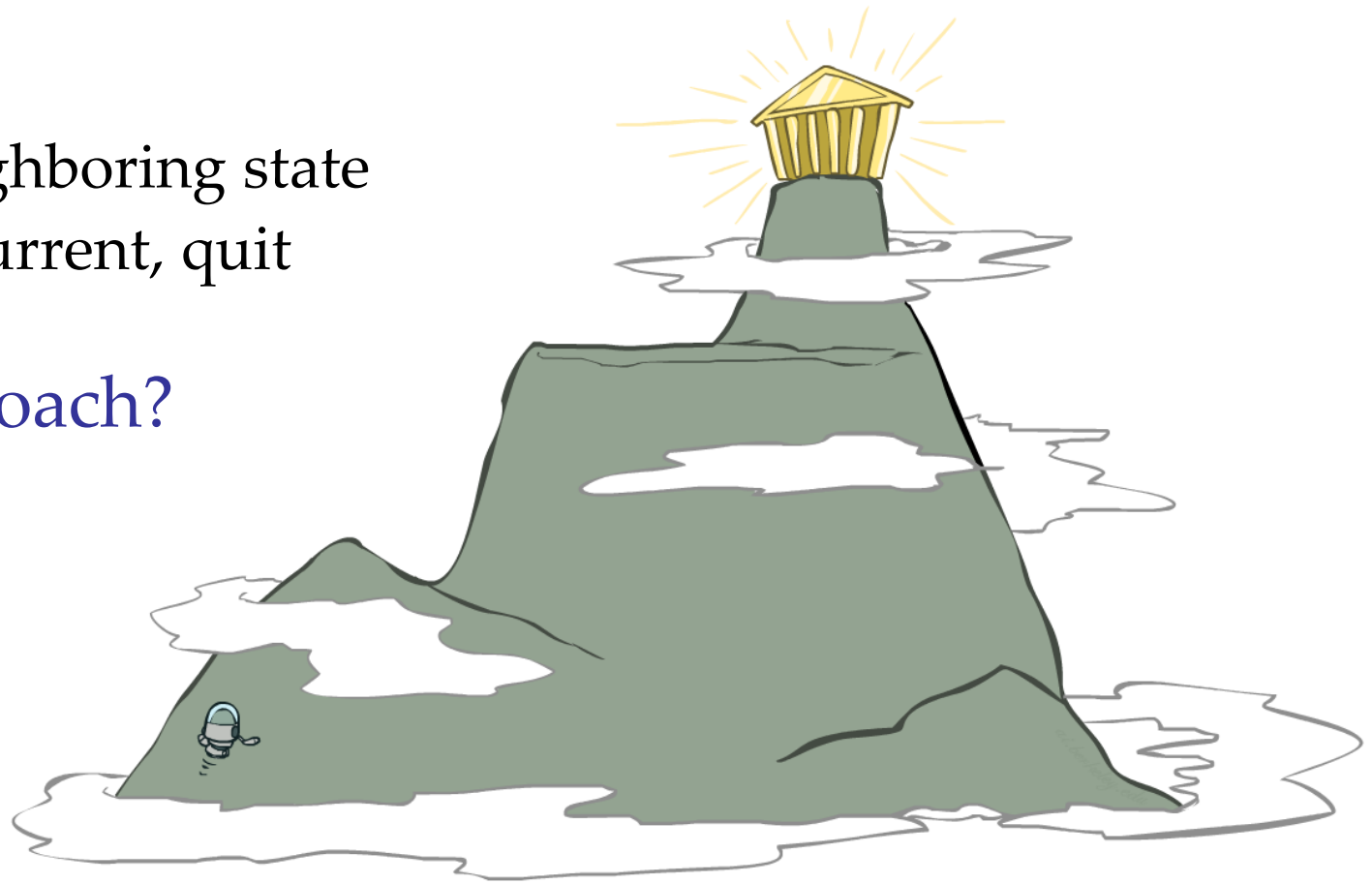


Hill Climbing Diagram

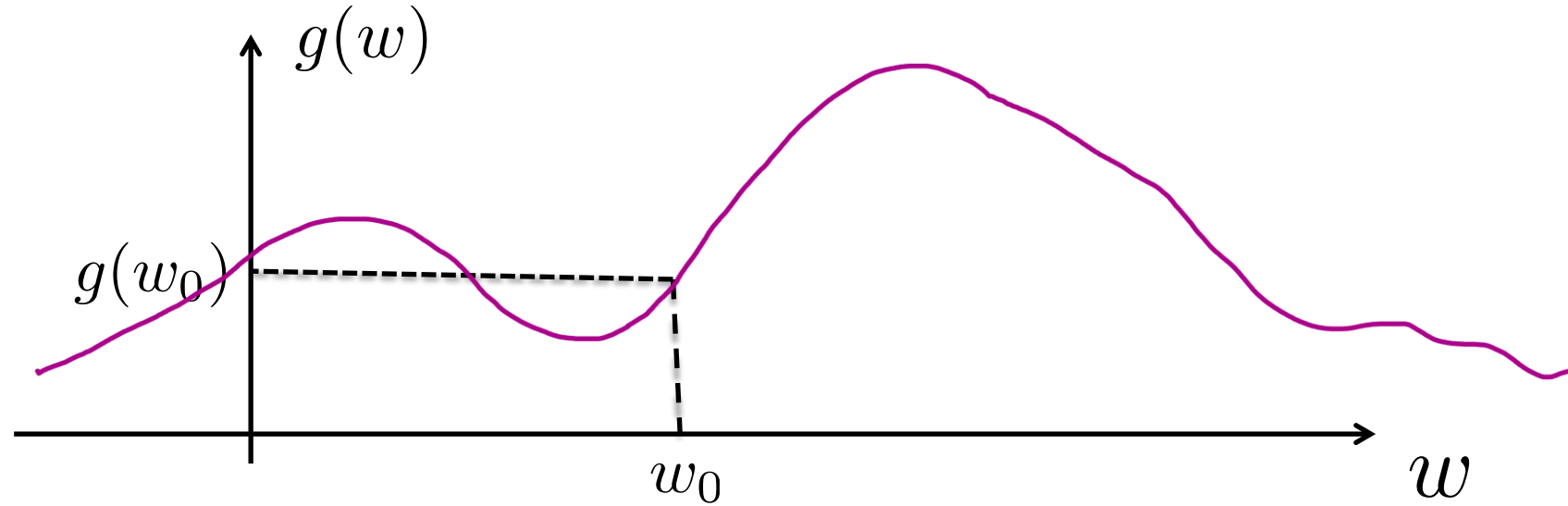


Hill Climbing

- Simple, general idea:
 - Start wherever
 - Repeat: move to the best neighboring state
 - If no neighbors better than current, quit
- What's bad about this approach?
- What's good about it?

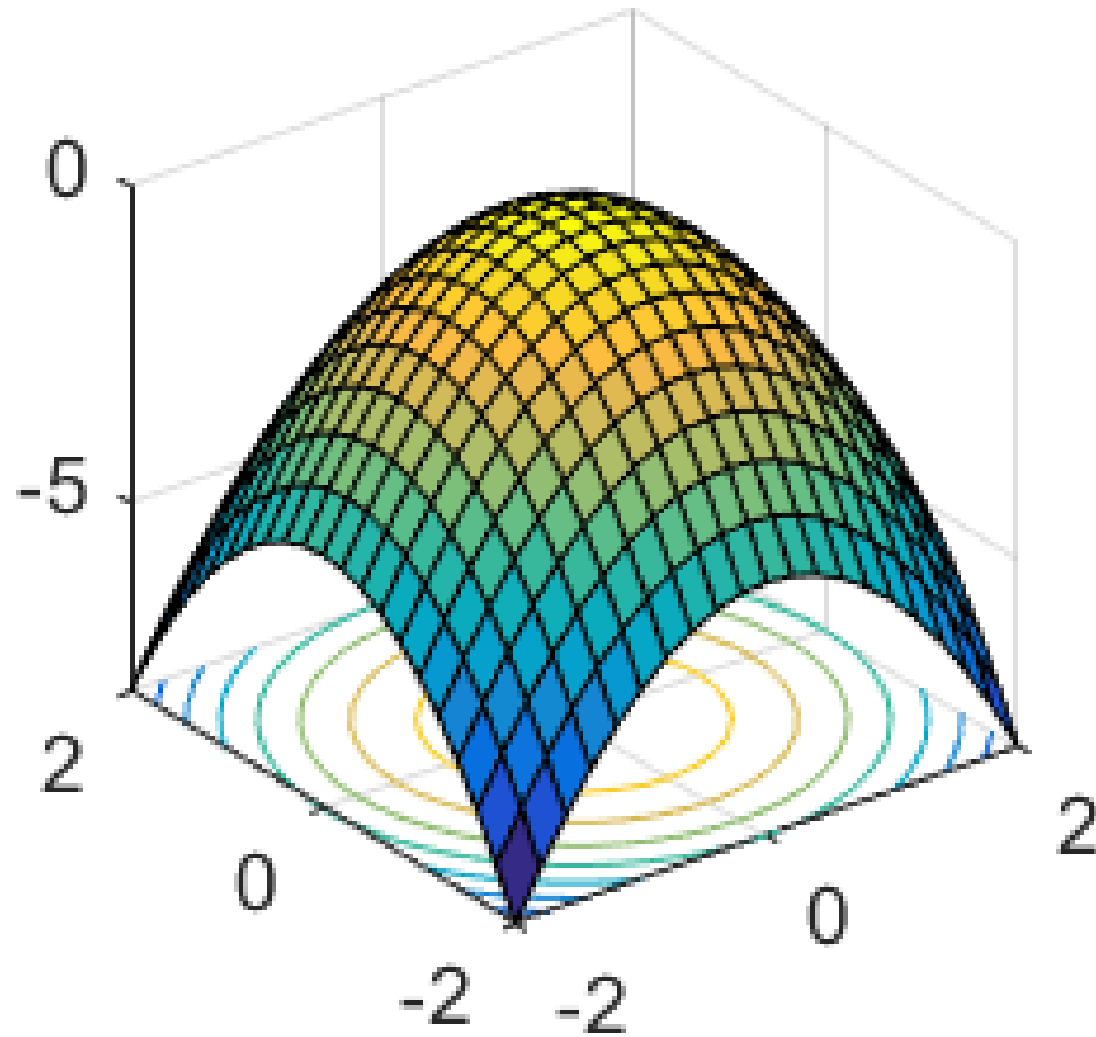


1-D Optimization



- Could evaluate $g(w_0 + h)$ and $g(w_0 - h)$
 - Then step in best direction
- Or, evaluate derivative: $\frac{\partial g(w_0)}{\partial w} = \lim_{h \rightarrow 0} \frac{g(w_0 + h) - g(w_0 - h)}{2h}$
 - Tells which direction to step into

2-D Optimization



Gradient Ascent

- Perform update in uphill direction for each coordinate
- The steeper the slope (i.e. the higher the derivative) the bigger the step for that coordinate
- E.g., consider: $g(w_1, w_2)$

- Updates:

$$w_1 \leftarrow w_1 + \alpha * \frac{\partial g}{\partial w_1}(w_1, w_2)$$

$$w_2 \leftarrow w_2 + \alpha * \frac{\partial g}{\partial w_2}(w_1, w_2)$$

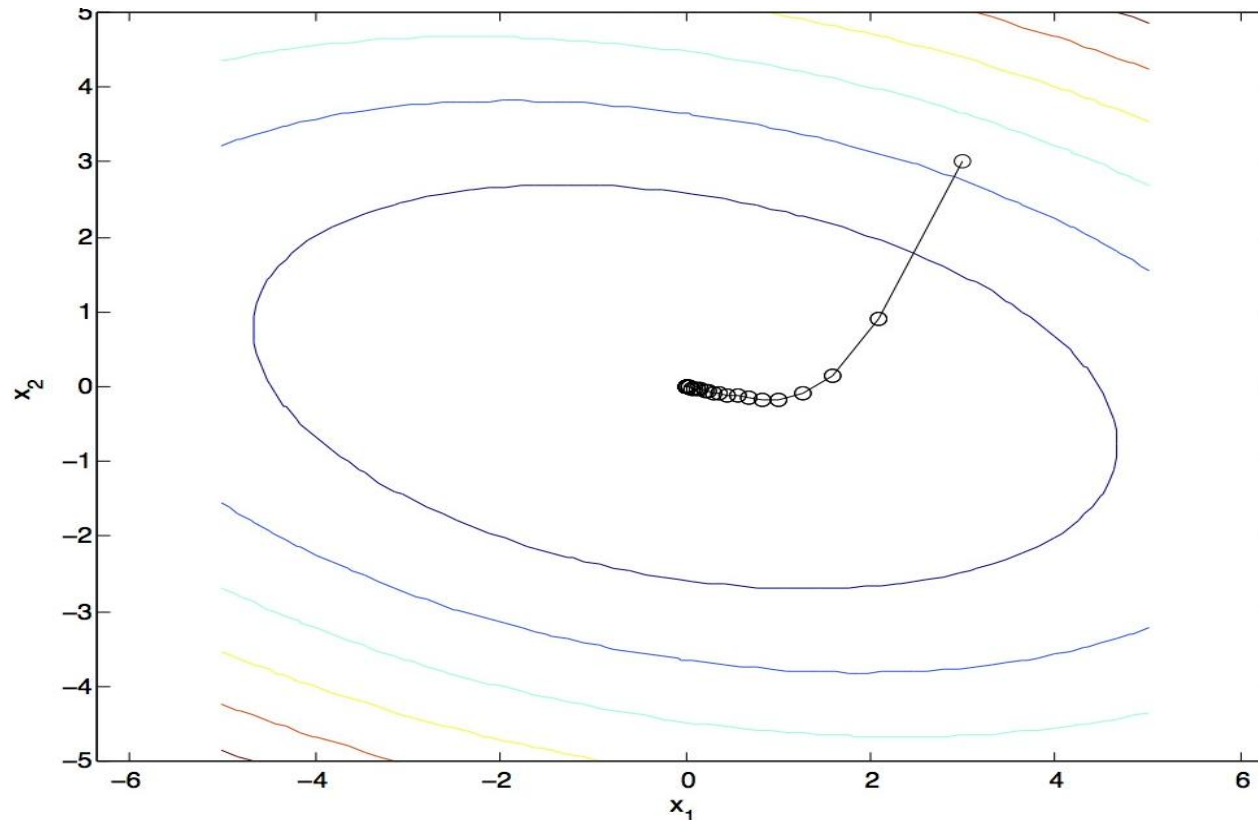
- Updates in vector notation:

$$w \leftarrow w + \alpha * \nabla_w g(w)$$

$$\text{with: } \nabla_w g(w) = \begin{bmatrix} \frac{\partial g}{\partial w_1}(w) \\ \frac{\partial g}{\partial w_2}(w) \end{bmatrix} = \text{gradient}$$

Gradient Ascent

- Idea:
 - Start somewhere
 - Repeat: Take a step in the gradient direction



What is the Steepest Direction?

$$\max_{\Delta: \Delta_1^2 + \Delta_2^2 \leq \varepsilon} g(w + \Delta)$$



- First-Order Taylor Expansion:
$$g(w + \Delta) \approx g(w) + \frac{\partial g}{\partial w_1} \Delta_1 + \frac{\partial g}{\partial w_2} \Delta_2$$

- Steepest Ascent Direction:
$$\max_{\Delta: \Delta_1^2 + \Delta_2^2 \leq \varepsilon} g(w) + \frac{\partial g}{\partial w_1} \Delta_1 + \frac{\partial g}{\partial w_2} \Delta_2$$

- Recall:
$$\max_{\Delta: \|\Delta\| \leq \varepsilon} \Delta^\top a \rightarrow \Delta = \varepsilon \frac{a}{\|a\|}$$

- Hence, solution:
$$\Delta = \varepsilon \frac{\nabla g}{\|\nabla g\|}$$
 Gradient direction = steepest direction!

$$\nabla g = \begin{bmatrix} \frac{\partial g}{\partial w_1} \\ \frac{\partial g}{\partial w_2} \end{bmatrix}$$

Gradient in n dimensions

$$\nabla g = \begin{bmatrix} \frac{\partial g}{\partial w_1} \\ \frac{\partial g}{\partial w_2} \\ \dots \\ \frac{\partial g}{\partial w_n} \end{bmatrix}$$

Optimization Procedure: Gradient Ascent

```
■ init  $w$   
■ for  $iter = 1, 2, \dots$   

$$w \leftarrow w + \alpha * \nabla g(w)$$

```

- α : learning rate --- tweaking parameter that needs to be chosen carefully
- How? Try multiple choices
 - Crude rule of thumb: update changes w about 0.1 – 1 %

Batch Gradient Ascent on the Log Likelihood Objective

$$\max_w ll(w) = \max_w \underbrace{\sum_i \log P(y^{(i)} | x^{(i)}; w)}_{g(w)}$$

- `init` w
- `for` `iter` = 1, 2, ...

$$w \leftarrow w + \alpha * \sum_i \nabla \log P(y^{(i)} | x^{(i)}; w)$$

What will gradient ascent do?

$$w \leftarrow w + \alpha * \sum_i \nabla \log P(y^{(i)} | x^{(i)}; w)$$

$$P(y^{(i)} | x^{(i)}; w) = \frac{e^{w_{y^{(i)}} \cdot f(x^{(i)})}}{\sum_y e^{w_y \cdot f(x^{(i)})}}$$

$$\nabla w_{y^{(i)}} f(x^{(i)}) - \nabla \log \sum_y e^{w_y f(x^{(i)})}$$

adds f to the correct
class weights

$$\frac{1}{\sum_y e^{w_y f(x^{(i)})}} \sum_y \left(e^{w_y f(x^{(i)})} [0^T f(x^{(i)})^T 0^T]^T \right)$$

$$\text{for } y' \text{ weights: } \frac{1}{\sum_y e^{w_y f(x^{(i)})}} e^{w_{y'} f(x^{(i)})} f(x^{(i)})$$

$P(y' | x^{(i)}; w) f(x^{(i)})$ subtracts f from y' weights in proportion to
the probability current weights give to y'

Stochastic Gradient Ascent on the Log Likelihood Objective

$$\max_w ll(w) = \max_w \sum_i \log P(y^{(i)} | x^{(i)}; w)$$

Observation: once gradient on one training example has been computed, might as well incorporate before computing next one

- `init  $w$`
- `for iter = 1, 2, ...`
 - `pick random  $j$`

$$w \leftarrow w + \alpha * \nabla \log P(y^{(j)} | x^{(j)}; w)$$

Mini-Batch Gradient Ascent on the Log Likelihood Objective

$$\max_w ll(w) = \max_w \sum_i \log P(y^{(i)} | x^{(i)}; w)$$

Observation: gradient over small set of training examples (=mini-batch) can be computed in parallel, might as well do that instead of a single one

- `init` w
- `for` $iter = 1, 2, \dots$
 - pick random subset of training examples J

$$w \leftarrow w + \alpha * \sum_{j \in J} \nabla \log P(y^{(j)} | x^{(j)}; w)$$

How about computing all the derivatives?

- We'll talk about that once we covered neural networks, which are a generalization of logistic regression